ML Model Drift Early-Warning System

B. Tech. Project Report

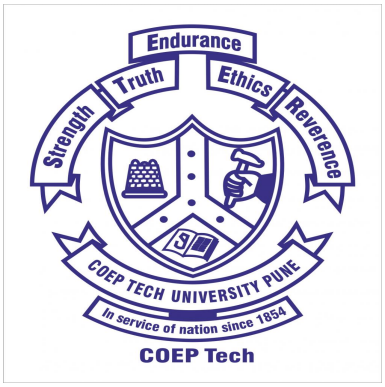
Submitted by

Akshay Naphade	**612303014**
Rushikesh Kadepurkar	**612303077**
Parth Tatpuje	**612303081**
Vishnu Gurav	**642303005**

Under the guidance of

Prof. Sunil Bhirud

COEP Technological University, Pune



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

COEP TECHNOLOGICAL UNIVERSITY

(COEP TECH), PUNE - 5

November 2026

DEPARTMENT OF COMPUTER SCIENCE &

ENGINEERING,

COEP TECHNOLOGICAL UNIVERSITY, PUNE - 5

CERTIFICATE

Certified that this project titled, "ML Model Drift Early-Warning System" has been successfully completed by

Akshay Naphade	**612303014**
Rushikesh Kadepurkar	**612303077**
Parth Tatpuje	**612303081**
Vishnu Gurav	**642303005**

and is approved for the partial fulfillment of the requirements for the degree of "B. Tech. Computer Engineering".

SIGNATURE
Prof. Sunil Bhirud
Project Guide
Department of CSE
COEP Tech Pune,
Shivajinagar, Pune - 5.

SIGNATURE
Dr. Pradip Deshmukh
Head
Department of CSE
COEP Tech Pune,
Shivajinagar, Pune - 5.

Plagiarism Report

[On this page, add the scanned copy of the plagiarism percentage page signed by the Project Guide.]

Acknowledgement

We express our sincere gratitude to our project guide, Prof. Sunil Bhirud, for his guidance, encouragement and critical feedback throughout this project. His insistence on checking every claim against evidence shaped the way this work was evaluated. We thank Dr. Pradip Deshmukh, Head of the Department of Computer Science and Engineering, and the faculty and staff of COEP Technological University, Pune, for providing the facilities and support needed to complete this work.

We also thank the maintainers of OpenML, scikit-learn, NumPy, SciPy, pandas, matplotlib and Streamlit, whose open-source tools and public datasets made this project possible.

Use of AI tools. An AI assistant (Claude, Anthropic) was used extensively in this project: to help implement and debug the experiment code, to generate figures and diagrams from the saved experiment outputs, and to draft the text of this report, which we then reviewed and revised. All experimental results were produced by the project's code and checked by us, and we take full responsibility for the content of this report.

Akshay Naphade

Rushikesh Kadepurkar

Parth Tatpuje

Vishnu Gurav

Abstract

Machine learning models deployed in production lose accuracy when the data they see drifts away from the data they were trained on. In many applications, such as credit, fraud or churn prediction, the true labels that would reveal this loss arrive days or weeks later. Drift detectors offer earlier, label-free signals, but most input shifts do not harm a given model, and alert thresholds tuned on one dataset often do not hold on another.

This project builds and evaluates an *ML Model Drift Early-Warning System* that tries to warn about *degradation*, not merely drift, before delayed labels can confirm it. We developed a benchmark that re-samples real tabular data from OpenML so that inputs drift from a known batch while every row keeps its true label; this lets us measure harm instead of assuming it. The system was developed over ten versions. Versions v1–v5 were exploratory studies on synthetic and natural data streams. Versions v6–v10 were pre-registered: every hypothesis, pass rule and alert threshold was frozen and fingerprinted in code before the confirmation data were downloaded, and each verdict was computed once by a script.

The first pre-registered design (v6), a learned risk calibrator over raw drift scores, failed all five of its hypotheses, because thresholds in dataset-specific units did not transfer: false alerts on unseen datasets ranged from 1.2 to 44.6 per 100 clean batches against a budget of 5. Expressing every signal relative to the stream's own burn-in period fixed this on benchmark streams. The resulting *drift-watch* tier (v7) warned before the accuracy drop on 87% of harmful gradual shifts on six fresh datasets, a median of six batches ahead, at 4.4 false alerts per 100 clean batches; it failed on natural streams, where drift was already under way when monitoring began. A second *harm* tier (v8) estimates the model's accuracy from an

importance-weighted labeled reference set. On five further fresh datasets it cut alarms on harmless shifts from 97% to 34% and separated harmful from harmless shifts with an AUROC of 0.78 (95% CI 0.70–0.86), just below the pre-registered target of 0.80 but clearly above confidence-based baselines (0.56–0.59) on the same data. On a later set of six datasets the same rule exceeded the false-alert budget (12.7 per 100). Two follow-up experiments, nonlinear harm estimators (v9) and stricter alerting rules (v10), did not improve on it and were not adopted.

The project delivers the benchmark and monitoring code, frozen configurations and verdict scripts, a Streamlit dashboard that replays monitored streams batch by batch with delayed labels, a continuous-integration pipeline, and a research paper. Its main lessons are that alert thresholds must be expressed in units that are comparable across streams, and that telling harmful from harmless drift without labels needs a labeled reference sample.

Keywords: concept drift, dataset shift, model monitoring, delayed labels, performance estimation, importance weighting, pre-registration, MLOps

Contents

Acknowledgement **iii**

Abstract **iv**

List of Tables **xii**

List of Figures **xiv**

List of Symbols and Abbreviations **xv**

1 Introduction **1**

1.1 Background 1

1.2 Motivation 2

1.3 Problem Statement 3

1.4 Objectives 3

1.5 Scope 4

1.6 Contributions 4

1.7 Organisation of the Report 5

2 Literature Survey **6**

2.1 Concept Drift and Dataset Shift 6

2.2 Drift Detection 7

2.2.1 Detectors that need labels 7

2.2.2 Label-free detectors 7
2.3 Label-Free Performance Estimation 7
2.4 Delayed Labels 8
2.5 Pre-Registration and Reproducibility 9
2.6 Summary and Research Gap 9

3 Problem Definition and Requirements 11
3.1 Formal Problem Setting 11
3.1.1 Degradation event 11
3.1.2 Episodes and outcomes 12
3.1.3 Warning tiers 12
3.1.4 False alerts 13
3.1.5 Separation of harm 13
3.2 Functional Requirements 13
3.3 Non-Functional Requirements 15
3.4 Hardware and Software Requirements 16
3.5 Users and Use Cases 16

4 System Design 18
4.1 System Architecture 18
4.2 Benchmark Design 19
4.2.1 Datasets and data groups 22
4.3 Label-Free Signals 23
4.4 The Two-Tier Monitor 24
4.4.1 Tier 1: drift watch (v7) 24
4.4.2 Tier 2: harm tier (v8) 25
4.4.3 Threshold selection 25
4.5 Evaluation Protocol and Pre-Registration 26

4.6 Software Design 27
4.6.1 Dashboard design 27

5 Implementation **29**
5.1 Tools and Technologies 29
5.2 Development History: Ten Versions 29
5.2.1 v1: the original prototype's risk calibrator 30
5.2.2 v2: redesigned scenarios and a state target 30
5.2.3 v3: delay-aware confidence-based estimation 31
5.2.4 v4: delay-anchored accuracy estimation 31
5.2.5 v5: harmless versus harmful shifts 32
5.2.6 v6–v10: pre-registered experiments 33
5.3 Code Structure 33
5.4 Key Implementation Details 34
5.4.1 Enforcing label delay 34
5.4.2 Per-batch monitoring 35
5.4.3 Pre-registration in code 35
5.4.4 Parallel extraction 36
5.5 Dashboard 36
5.6 Testing and Continuous Integration 40
5.7 Reproducibility 41
5.8 Cleaning Up the Prototype 41

6 Results and Discussion **43**
6.1 v6: The Original Design Fails 43
6.1.1 Benchmark gate 43
6.1.2 Pre-registered verdict 44
6.1.3 Secondary settings 45

6.2 Why v6 Failed: Thresholds in Raw Units Do Not Transfer 46
6.3 v7: The Drift-Watch Tier 47
6.3.1 Natural streams 49
6.4 v8: The Harm Tier 49
6.5 Same-Data Baselines (Post Hoc) 51
6.6 Why Harmless Shifts Still Alert 52
6.7 Pre-Registered Follow-Ups That Failed 53
6.8 Ablations 54
6.8.1 Two families of signals 54
6.8.2 Which harm estimator 55
6.8.3 Why most shifts are harmless 56
6.9 Discussion 57
6.9.1 Lessons 57
6.9.2 Threats to validity 57
6.9.3 Limitations 58

7 Conclusion and Future Work 59
7.1 Conclusion 59
7.2 Future Work 60

A Pre-Registered Hypotheses 62

B Datasets 64

C Reproducing the Results 66
C.1 Environment 66
C.2 Verdicts, ablations and post hoc analyses 66
C.3 Dashboard, tests, figures 66
C.4 Frozen files 67

Bibliography **71**

List of Tables

2.1 Families of drift-monitoring methods and their limits in the delayed-label setting. 9

3.1 Hardware and software used. 16

4.1 Data groups and their use (65 episodes per dataset). . . . 23
4.2 Label-free signals computed per batch. 23

5.1 v2 controlled study, label delay $L = 2$, 250 held-out synthetic episodes. 31
5.2 Main modules of the code base. 34

6.1 v6 pre-registered hypotheses on the held-out datasets. Overall verdict: FAILURE. 45
6.2 v6 secondary settings with 20% of labels (held-out): T2 on harmful shifts and false alerts per 100 clean batches. 46
6.3 v7 pre-registered hypotheses on six fresh datasets. Overall verdict: PARTIAL SUCCESS. 48
6.4 v8 pre-registered hypotheses on five fresh datasets. Overall verdict: PARTIAL SUCCESS. 50
6.5 Each normalised signal used alone (v7 confirmation data, post hoc). 55
6.6 Harm-estimator variants on the v8 confirmation data (thresholds by the separation rule). 56

List of Figures

3.1 Warning tiers on the batch axis, for onset $s = 12$, event $e = 20$ and label delay $L = 5$. T1 ends when accuracy drops; T2 ends when delayed labels could confirm the drop. 13

3.2 Use-case diagram of the Drift Early-Warning System. . . . 17

4.1 Layered architecture of the ML Model Drift Early-Warning System. 19

4.2 Flowchart of benchmark episode generation. 21

4.3 Per-batch data flow of the two-tier monitor and the confirmation by delayed labels. 24

4.4 Pre-registration workflow used for every experiment from v6 onward. 26

4.5 Software modules and their dependencies. 27

4.6 Data flow of the Streamlit dashboard. 28

5.1 The ten versions of the system and what each showed. . . 30

5.2 v4 accuracy nowcast error (mean absolute error in accuracy points) on 8 development streams (leave-one-dataset-out) and 5 held-out streams, by label delay. 32

5.3 Dashboard: Overview page. 37

5.4 Dashboard: Live monitor replaying a harmful gradual shift on an unseen dataset (2dplanes, label delay 5). Alerts before the shift at batch 12 count as false alerts in the evaluation. 38
5.5 Dashboard: Verdicts page. 39
5.6 Dashboard: "Why it works" page with the ablation findings. 40
5.7 Continuous-integration pipeline. 41

6.1 Share of development shift episodes that harmed the model, by maximum shift strength. The pre-registered gate required at least 25%. 44
6.2 v6 per held-out dataset: share of harmful shifts warned before the drop (T1) and before labels confirm (T2). 45
6.3 False alerts per 100 clean batches with thresholds chosen on development data, on development and on unseen data (v6 held-out data for the v6 methods; v7 and v8 confirmation data for the drift watch and the harm tier). Dashed line: budget of 5. 47
6.4 v7 confirmation data: early warning (T1), pre-confirmation warning (T2) and harmless-shift alarms of each method. . 48
6.5 v7 per confirmation dataset: T1 share of harmful shifts (n = number of harmful shifts). 49
6.6 v8 harmful-versus-harmless AUROC per confirmation dataset (number of harmful / harmless shifts in brackets). 50

6.7 A harmful gradual shift on an unseen dataset (credit default, $L = 5$). Top: true accuracy (hidden from the monitor) and the label-free estimate; dotted line: 5 points below baseline. Bottom: tier scores divided by their thresholds; above 1 is an alert. 51
6.8 Harmful-versus-harmless AUROC of label-free signals on the v8 confirmation data (post hoc), with 95% bootstrap intervals. 52
6.9 Post hoc operating curves on the v8 confirmation data; each point is a threshold. Circles: frozen thresholds. Square: the stricter v10 rule, frozen on development data and evaluated on six different datasets. 53
6.10 Harmless-shift alarms against pre-confirmation warnings for the frozen tiers and the v10 rule; FA = false alerts per 100 clean batches. 54
6.11 Early warning versus harm separation of each signal used alone (v7 confirmation data). 55
6.12 Share of shift episodes that harmed the model, by data group and by importance of the shifted feature. 56

List of Symbols and Abbreviations

Symbol	Meaning
f	Frozen (monitored) binary classifier
R	Labeled reference set (2000 rows) on which f was fitted
B_t	Batch number t of the monitored stream (300 rows)
L	Label delay in batches; labels of batch b are usable from batch $b + \max(L, 1)$
s	Batch at which the controlled intervention (shift) starts
e	Batch at which the accuracy event starts
$\lambda(t)$, λ_{max}	Shift strength at batch t and its maximum (1.5)
z_j	Feature j standardised with reference statistics
σ	Random shift direction, ± 1
x_t	Value of a monitoring signal at batch t
med, MAD	Median and median absolute deviation
g_t	Domain classifier separating reference rows from batch B_t
ρ_i, w_i	Odds of reference row i under g_t, and its capped importance weight
c_i	Correctness of f on reference row i (1 if correct)
$\hat{a}_t$	Label-free (importance-weighted) estimate of the accuracy of f on B_t

Symbol	Meaning
h_t	Harm score: estimated accuracy drop since the burn-in
T1, T2, T3	Warning tiers: predictive, pre-confirmation, confirmed
FA	False alerts per 100 clean batches
AUROC	Area under the receiver operating characteristic curve
CBPE	Confidence-based performance estimation
CI	Confidence interval (95% bootstrap unless stated) or continuous integration
DEWS	Drift Early-Warning System (the original project prototype)
IW	Importance weighting
KS	Kolmogorov–Smirnov statistic
MMD	Maximum mean discrepancy
PSI	Population stability index
RWS	Relevance-weighted shift
kNN	k-nearest neighbours

Chapter 1

Introduction

1.1 Background

A machine learning classifier is trained once, on data collected in the past, and is then deployed to make predictions on new data for months or years. The world that generates that data keeps changing: customers change their behaviour, sensors age, upstream software changes how a feature is computed, and new kinds of cases appear. When the distribution of the incoming data moves away from the training distribution, the model may start making more mistakes. This phenomenon is known as *dataset shift* or *concept drift* [8, 21].

The obvious way to notice a drop in accuracy is to compare the model's predictions with the true outcomes. In many real applications, however, the true outcome (the *label*) arrives only after a long delay. Whether a loan defaults is known after months; whether a transaction was fraudulent is known after a chargeback; whether a customer churned is known at the end of a billing cycle. During this delay the model keeps making decisions, and a silent degradation can cause real damage before anyone sees it.

Production monitoring tools therefore watch *label-free* signals: how the distribution of each input feature compares with the reference data, how

confident the model is, and how its predictions are distributed [23, 2]. These signals are available immediately. Two practical problems limit them:

1. **Most shifts are harmless.** A change in the inputs does not necessarily change the model's accuracy [22]. A drift detector that alarms on every change produces many alarms that do not matter, and operators learn to ignore it.

2. **Thresholds do not transfer.** An alert threshold is normally tuned on historical data from one application. Whether the same threshold still means the same thing on another dataset, or even on the same dataset next year, is rarely tested.

1.2 Motivation

The project started from an existing prototype, the *Drift Early-Warning System* (DEWS). It combined several drift statistics (Kolmogorov–Smirnov tests, the population stability index, an embedding-based maximum mean discrepancy and a confidence component) into a single fused drift score, compared it with an adaptive threshold, and forecast when the score would cross the threshold. The prototype also contained a FastAPI service, a SQLite store, and a large infrastructure stack (Kafka, Celery, Redis and Grafana).

When we evaluated this prototype carefully, it did not do what its name promised. On held-out controlled episodes its learned risk calibrator detected 0 of 20 harmful events, and the fused drift score detected some events only after the accuracy had already dropped (Section 5.2.1). The infrastructure made the system look like a product, but there was no evidence

that its warnings were early or that they meant degradation.

This motivated a change of focus. Instead of adding more components, we asked a precise question and tried to answer it honestly, with experiments designed so that a negative answer would be visible.

1.3 Problem Statement

> Given a frozen binary classifier, a stream of unlabeled batches, and true labels that arrive only after a delay of L batches, build a monitor that warns about a sustained loss of accuracy (a) before the accuracy actually drops, and (b) before the delayed labels could confirm the drop, while raising at most 5 false alerts per 100 batches of unchanged data, and while separating shifts that harm the model from shifts that do not.

The precise definitions of a degradation event, the warning tiers and the false-alert budget are given in Chapter 3.

1.4 Objectives

1. Build a realistic benchmark on real tabular datasets in which input shifts start at a known batch while every row keeps its true label, so that whether a shift harmed the model can be measured rather than assumed.
2. Design label-free monitoring signals and an alerting scheme that warn early about harmful drift within a fixed false-alert budget.
3. Separate harmful from harmless shifts without using the labels of the monitored batches.

4. Evaluate every claim with pre-registered hypotheses, frozen thresholds and one-time runs on datasets that were never used during design, and report failures as clearly as successes.

5. Deliver a working, reproducible product: the monitoring library, experiment scripts, a dashboard that replays monitored streams, automated tests and continuous integration, and a research paper.

1.5 Scope

In scope. One frozen binary classifier on tabular data (logistic regression as the primary model, gradient boosting as a robustness check); batches of 300 rows; label delays of 0, 2, 5 and 10 batches; controlled covariate shifts and label changes built from real data; natural row-order streams as a reality check; and an offline replay dashboard.

Out of scope. Automated retraining, distributed stream processing, production throughput and failover, image or text models, and claims about arbitrary concept drift. The Kafka, Celery, Redis and Grafana stack of the original prototype was removed from the repository because it did not contribute to the research question.

1.6 Contributions

1. A re-sampling benchmark that turns real OpenML datasets into monitored streams with gradual, abrupt and irrelevant-feature shifts and label changes with a known onset, in which harm is measured from true labels.

2. Evidence that drift thresholds expressed in raw, dataset-specific units do not transfer across datasets, and that expressing every signal rel-

ative to a stream's own burn-in period largely fixes this: the resulting drift watch held its false-alert budget on unseen benchmark data, though not on natural streams.

3. A label-free harm tier based on importance-weighted reference accuracy that cuts alarms on harmless shifts from 97% to 34% and separates harm better than confidence-based estimates on the same data.

4. A complete record of pre-registered negative results: a failed first design, a failed second tier, a benchmark gate that failed, and two follow-up attempts that were not adopted.

5. A reproducible software product: library, frozen configurations, verdict scripts, dashboard, 85 unit tests, continuous integration, and a research paper prepared for IEEE CONECCT and arXiv.

1.7 Organisation of the Report

Chapter 2 surveys related work on drift detection, label-free performance estimation, delayed labels and pre-registration. Chapter 3 defines the problem formally and lists the requirements. Chapter 4 presents the system architecture, the two-tier monitor, the benchmark and the evaluation protocol. Chapter 5 describes the implementation, including the exploratory versions v1–v5, the code structure, the dashboard and the testing infrastructure. Chapter 6 reports and discusses the results of the pre-registered experiments v6–v10, the ablations and the post hoc analyses. Chapter 7 concludes and outlines future work. The appendices list every pre-registered hypothesis, the datasets used, and the commands needed to reproduce the results.

Chapter 2

Literature Survey

This chapter reviews the work that the project builds on: concept drift detection, dataset shift, label-free estimation of model performance, learning with delayed labels, and pre-registration as a method for trustworthy evaluation. It ends with the research gap that the project addresses.

2.1 Concept Drift and Dataset Shift

Gama et al. [8] and Lu et al. [15] survey concept drift: changes over time in the joint distribution $P(X, Y)$ of inputs X and labels Y. It is useful to separate two cases [21]:

- **Covariate shift**: $P(X)$ changes while $P(Y \mid X)$ stays the same. The model's decision rule is still correct where it was correct, but the data may move into regions where the model was weak.
- **Concept shift**: $P(Y \mid X)$ changes, so the same input now has a different correct label. No input-only signal can see this; only labels reveal it.

Label shift, where $P(Y)$ changes and $P(X \mid Y)$ stays the same, is treated by Lipton et al. [14].

2.2 Drift Detection

2.2.1 Detectors that need labels

Classical detectors monitor the error stream of a model. DDM [7] signals a warning and then a drift when the error rate rises significantly above its historical minimum. ADWIN [1] keeps an adaptive window and cuts it when two sub-windows have significantly different means. Sequential tests such as CUSUM [19] accumulate evidence of a persistent change. All of these need the label of each prediction, so with delayed labels they react late by construction.

2.2.2 Label-free detectors

Label-free detection compares the distribution of the current inputs with a reference sample. Common statistics are the two-sample Kolmogorov–Smirnov (KS) test per feature [16], the population stability index (PSI) used in credit scoring, divergences between histograms, and the kernel maximum mean discrepancy (MMD) [10]. Rabanser et al. [22] compared such detectors systematically and noted an important limitation: a detected shift need not reduce accuracy. Open-source monitoring tools such as Evidently [5] and NannyML [17] package these statistics for practitioners.

2.3 Label-Free Performance Estimation

A second line of work estimates the model's accuracy directly, without labels:

- **Confidence-based estimation (CBPE)** [17] uses the model's predicted probabilities as the expected accuracy. It is exact only if the model is calibrated on the new data.

- **Average thresholded confidence (ATC)** [9] learns a confidence threshold on source data and predicts the target accuracy as the share of target examples above it.
- **Difference of confidences (DoC)** [12] regresses the accuracy change on the change in average confidence.
- **Disagreement** [13]: the rate at which two independently trained models disagree tracks their error rate under certain conditions.
- **Importance weighting** [24, 26]: under covariate shift, reweighting labeled source examples by the density ratio $p_{target}(x)/p_{source}(x)$ gives an unbiased estimate of any target expectation, including accuracy. The density ratio can be estimated with a probabilistic classifier that separates source from target samples.

Our harm tier (Section 4.4.2) uses importance weighting with a logistic domain classifier, applied to the model's correctness on the labeled reference set.

2.4 Delayed Labels

Learning and evaluation with delayed or partially missing labels are studied by Grzenda et al. [11] and Umer and Polikar [27]. In the monitoring setting the key point is causality: at batch t, only the labels of batches $b \leq t - \max(L, 1)$ may be used. Any evaluation that ignores this leaks future information and overstates how early a monitor can warn. Our replay protocol enforces this rule and tests it (Section 5.6).

2.5 Pre-Registration and Reproducibility

Pre-registration means fixing hypotheses, analysis methods and success criteria before seeing the data that will judge them [18]. It reduces the freedom to choose, after the fact, the analysis that looks best. Van Miltenburg et al. [28] discuss pre-registration for machine learning and natural language processing research. Sculley et al. [23] and Breck et al. [2] argue that production ML systems need explicit, testable monitoring. We apply pre-registration to the evaluation of a monitoring system, including its alert thresholds, and we use public datasets from OpenML [29] so that every result can be reproduced.

2.6 Summary and Research Gap

Table 2.1 summarises the main families of methods.

Table 2.1: Families of drift-monitoring methods and their limits in the delayed-label setting.

Family	Examples	Limitation for early warning with delayed labels
Error-stream detectors	DDM, ADWIN, CUSUM	Need labels; with delay L they react at least L batches late
Input-distribution tests	KS, PSI, MMD, fused scores	Detect any change, harmful or not; thresholds in dataset-specific units
Confidence-based estimators	CBPE, ATC, DoC	Assume calibration on new data; can stay confident while wrong
Disagreement	auxiliary models	Moves with any shift that the auxiliary models notice
Importance weighting	density-ratio reweighting	Valid only under covariate shift; needs a labeled reference sample

From the survey we identified three gaps:

1. Most evaluations report whether drift is detected, not whether a warn-

ing comes *before* the accuracy drop or *before* delayed labels could reveal it.

2. Alert thresholds are rarely tested on datasets other than the ones they were chosen on.

3. Few studies measure, on real data with known ground truth, how often a drift alarm corresponds to actual harm.

The project addresses these gaps with a benchmark in which harm is measured, warning tiers defined relative to label arrival, and pre-registered confirmation on datasets never used during design.

Chapter 3

Problem Definition and Requirements

3.1 Formal Problem Setting

A frozen binary classifier f is trained on a labeled reference set R and then scores a stream of batches $B_0, B_1, B_2, \ldots$ of 300 rows each. The monitor sees the rows of every batch and the predictions of f immediately. The true labels of batch b become usable at batch $b + \max(L, 1)$, where L is the label delay (primary setting $L = 5$). The first 8 batches after deployment form a *burn-in* period during which no alerts are raised; the monitor uses them to learn what normal looks like for this stream.

3.1.1 Degradation event

The baseline accuracy is the mean accuracy of f on batches 0–4. A *degradation event* begins at the first of two consecutive batches whose accuracy is at least 5 percentage points below the baseline. Requiring two consecutive batches avoids calling a single noisy batch an event.

3.1.2 Episodes and outcomes

An *episode* is one monitored stream of 45 batches. In controlled episodes an intervention starts at a known batch s. Each shift episode is then labelled from the true accuracy:

- **Harmful**: an event starts at or after s;
- **Harmless**: no event occurs;
- **Spurious**: an event starts before s (or occurs in a clean episode); spurious episodes are excluded from the main hypotheses and their number is reported.

3.1.3 Warning tiers

For an intervention at batch s and an event at batch e we distinguish three tiers (Fig. 3.1):

- **T1, predictive warning**: an alert in $[s, e)$, before the accuracy actually drops. The *lead time* is e minus the first such alert.
- **T2, pre-confirmation warning**: an alert in $[s, e + 1 + \max(L, 1))$, before delayed labels could confirm the event. This is the tier that matters most in practice: it is the information the monitor adds over simply waiting for labels.
- **T3, confirmed**: the first alert at or after confirmation; by then the labels already show the drop.

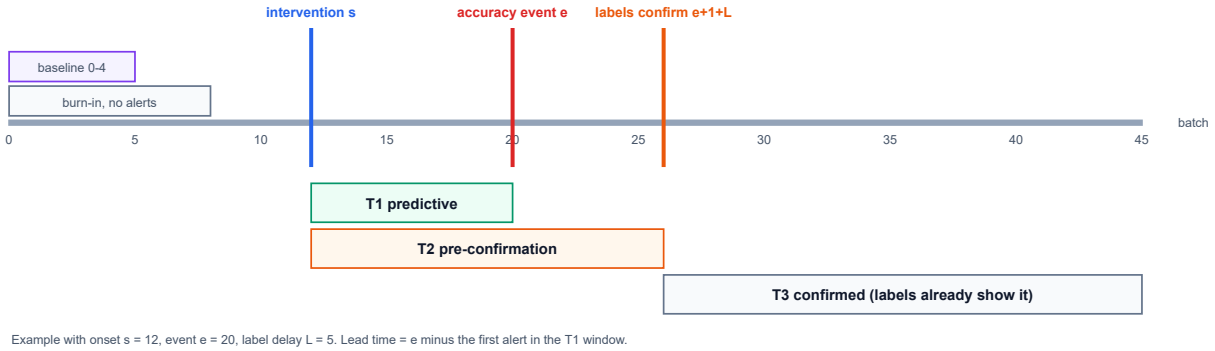


Figure 3.1: Warning tiers on the batch axis, for onset $s = 12$, event $e = 20$ and label delay $L = 5$. T1 ends when accuracy drops; T2 ends when delayed labels could confirm the drop.

3.1.4 False alerts

Clean batches are all batches of unshifted (clean) episodes and the pre-intervention batches of every other episode. The *false-alert rate* is the number of alerts on clean batches per 100 clean batches, with a budget of **5**. For harmless shifts, which never cause an event, we report the *harmless-alarm share*: the share of harmless episodes with any alert after onset.

3.1.5 Separation of harm

To measure whether a score separates harmful from harmless shifts, we take its mean over the first 3 batches after onset in each shift episode and compute the area under the ROC curve (AUROC) between harmful and harmless episodes. An AUROC of 0.5 means no separation; 1.0 means perfect separation.

3.2 Functional Requirements

FR1. **Stream construction.** Build monitored episodes from any binary tabular OpenML dataset: reference set, pool, 45 batches, controlled shifts with a known onset, and true labels for every row.

FR2. **Signal extraction.** For every batch, compute label-free signals: per-feature drift statistics, the fused DEWS drift score, relevance-weighted shift, positive-rate shift, confidence, entropy, and disagreement with auxiliary models.

FR3. **Delayed labels.** Release the labels of batch b only at batch $b + \max(L, 1)$; never let a score at batch t use a label that has not matured.

FR4. **Drift watch.** Normalise every signal against the stream's burn-in and raise a WATCH alert when the largest normalised signal exceeds the frozen threshold.

FR5. **Harm tier.** Estimate the model's current accuracy from the labeled reference set by importance weighting and raise a HARM alert when the estimated drop exceeds the frozen threshold.

FR6. **Confirmation.** When delayed labels arrive, confirm or clear the warning with the event rule.

FR7. **Evaluation.** Compute T1, T2, T3, lead times, false-alert rates, harmless-alarm shares and AUROC with bootstrap confidence intervals.

FR8. **Pre-registration.** Freeze thresholds and rules in versioned configuration files with SHA-256 fingerprints, and refuse to run a confirmation if a fingerprint does not match.

FR9. **Verdicts.** Apply the frozen pass rules mechanically and write a verdict file and report.

FR10. **Dashboard.** Replay a saved episode batch by batch with a chosen label delay, show both tiers and the delayed labels, explain which signals moved, and show all verdicts and ablations.

3.3 Non-Functional Requirements

NFR1. **Causality.** No score may use future labels or event outcomes; this is enforced in code and covered by unit tests.

NFR2. **Reproducibility.** Every episode is generated from a recorded seed; all models use fixed random states; bootstrap intervals use fixed generator seeds; a single command rebuilds and verifies the verdicts.

NFR3. **Honesty of evaluation.** Thresholds are chosen on development data only; confirmation data are downloaded only after the design is frozen; each confirmation is run once.

NFR4. **Performance.** Monitoring a batch must take well under a second on a laptop; the earlier replay pipeline measured about 0.08 seconds per batch.

NFR5. **Maintainability.** Code is linted (Ruff), unit-tested (pytest) and checked by continuous integration on Python 3.10, 3.11 and 3.12.

NFR6. **Usability.** The dashboard must be understandable to a non-specialist examiner in a short demonstration.

3.4 Hardware and Software Requirements

Table 3.1: Hardware and software used.

Item	Specification
Hardware	Laptop, Intel Core i7-12650H (10 cores, 16 threads), 16 GB RAM; no GPU needed
Operating system	Windows 11 (development); Ubuntu (continuous integration)
Language	Python 3.10–3.12 (development on 3.12)
Core libraries	NumPy 2.5.4, SciPy, pandas 2.3, scikit-learn 1.7.2, statsmodels, matplotlib
Dashboard	Streamlit 1.57+, Altair
Testing and CI	pytest 8, Ruff 0.17.4, GitHub Actions
Data source	OpenML [29], fetched with scikit-learn
Documents	LaTeX (IEEEtran for the paper, this template for the report), draw.io for diagrams
Legacy (optional)	FastAPI, uvicorn, MLflow, PostgreSQL drivers, in a separate environment

3.5 Users and Use Cases

Two kinds of users interact with the system (Fig. 3.2). An *ML engineer* uses the monitor and the dashboard to see whether a deployed model is drifting and whether the drift is likely to hurt it. A *researcher or examiner* reads the verdicts, reproduces them, and runs new pre-registered experiments. OpenML acts as an external system that supplies datasets.

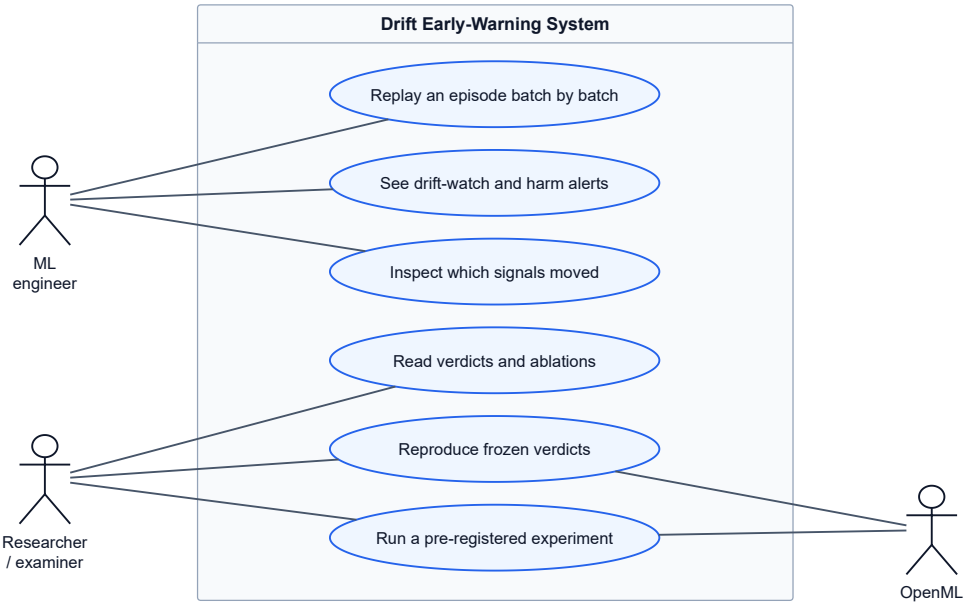


Figure 3.2: Use-case diagram of the Drift Early-Warning System.

Chapter 4

System Design

This chapter describes the design of the final system: its architecture, the benchmark that produces monitored streams with known ground truth, the label-free signals, the two warning tiers, and the protocol used to evaluate them. The history of how the design was reached is given in Chapter 5.

4.1 System Architecture

Fig. 4.1 shows the architecture as five layers.

- **Data.** Public tabular datasets from OpenML feed a benchmark generator, which builds monitored streams with a known intervention batch. Natural row-order streams are used as a reality check.
- **Model.** A frozen classifier f is fitted once on a labeled reference set R. Each incoming batch B_t is scored by f; its labels stay hidden.
- **Signals.** Label-free signals are computed per batch: input-shift statistics and model-behaviour statistics.
- **Monitor.** Two tiers turn the signals into alerts: a *drift watch* that is sensitive to any drift, and a *harm tier* that estimates whether the drift is lowering accuracy.

- **Outputs.** Alerts are confirmed or cleared when delayed labels arrive. Evaluation scripts compute the warning tiers and apply the pre-registered pass rules; the dashboard, paper and this report are generated from the saved results.

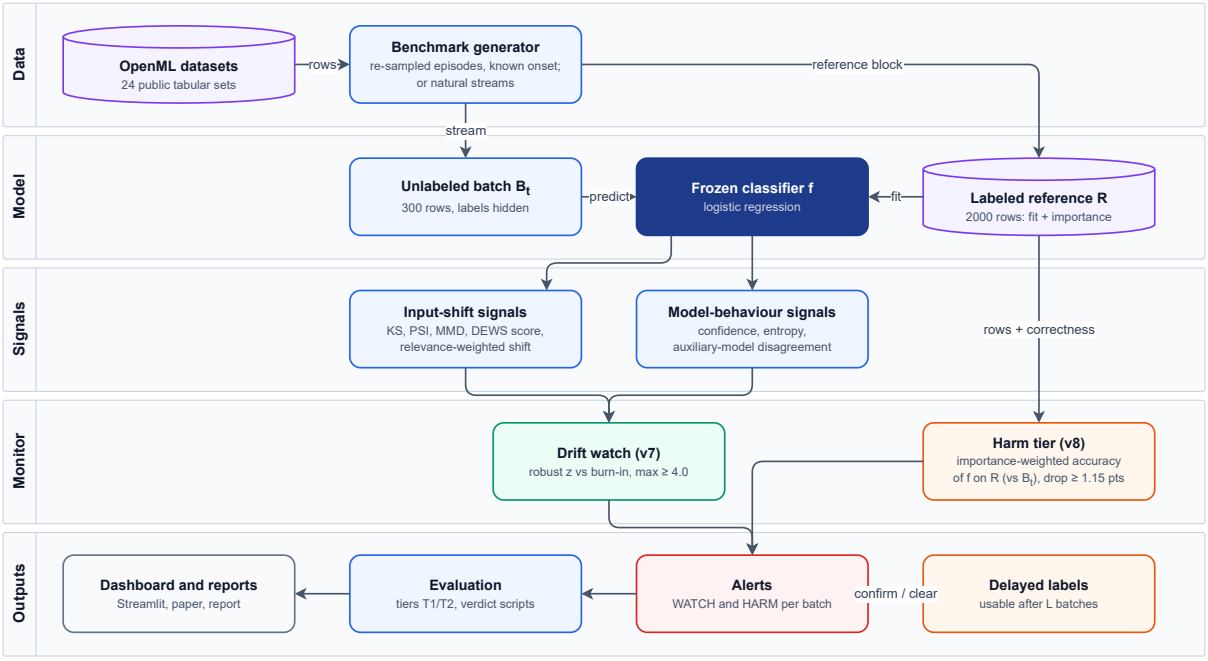


Figure 4.1: Layered architecture of the ML Model Drift Early-Warning System.

4.2 Benchmark Design

Evaluating an early-warning system needs ground truth: we must know when a shift started and whether it actually hurt the model. Synthetic generators provide this but are unrealistic (Section 5.2.2); natural streams are realistic but have no known onset. The benchmark combines the two by *re-sampling real rows*.

Each episode draws a contiguous block of a dataset (Fig. 4.2):

- a **reference** of 2000 rows: f is fitted on the first 1500, and the last 500 are held out to compute f's permutation feature importance;
- a **pool** of the next 20,000 rows. Because reference and pool come from

the same period, an unshifted stream has no built-in drift.

The stream has 45 batches of 300 rows sampled with replacement from the pool; *every row keeps its true label*, so the true accuracy of f on every batch is known. From an onset batch $s \in [10, 16]$:

- **Gradual or abrupt shift**: rows are sampled with weights proportional to $\exp(\lambda(t)\,\sigma\,z_j)$, where z_j is a standardised feature, $\sigma = \pm 1$ a random direction, and λ ramps linearly to 1.5 over 15 batches (gradual) or jumps to 1.5 (abrupt). The feature j is drawn from the more important or the less important half of features with equal probability, so that some shifts are designed to be irrelevant.
- **Label change**: labels are flipped with a probability that ramps to 0.3 for rows with $z_k > 0$, a change in $P(Y \mid X)$ that input-only signals cannot see.
- **Clean**: uniform sampling.

Each dataset contributes 10 clean, 30 gradual, 15 abrupt and 10 label-change episodes (65 episodes). Algorithm 1 summarises the construction.



Figure 4.2: Flowchart of benchmark episode generation.

Algorithm 1 Building one benchmark episode

Require: dataset (X, y), episode kind, seed

draw a block start where both classes are $\geq$5% of the fit rows and of the pool
$R \leftarrow$ next 2000 rows; fit f on the first 1500; permutation importance on the last 500
$P \leftarrow$ next 20,000 rows (pool); draw onset $s \in [10, 16]$
if kind is a shift **then**
    pick feature j (important or unimportant half) and sign σ
end if
for $t = 0, \ldots, 44$ **do**
    if kind is a shift and $t \geq s$ **then**
        sample 300 rows of P with weights $\propto \exp(\lambda(t)\, \sigma\, z_j)$
    else
        sample 300 rows of P uniformly
    end if
    if kind is a label change and $t \geq s$ **then**
        flip labels of rows with $z_k > 0$ with probability $0.3 \min(1, (t - s + 1)/15)$
    end if
end for
compute the accuracy of f on every batch; label the episode harmful, harmless, clean or spurious

4.2.1 Datasets and data groups

All datasets are binary (or reduced to majority class versus the rest) and use their first 20 non-constant features. Table 4.1 shows how they were used. The 13 v6 datasets had already been used as natural-order streams in the exploratory studies v1–v5; the confirmation groups of v7, v8 and v10 were downloaded only after the corresponding design was frozen and had never been used before. The full list is in Appendix B.

Table 4.1: Data groups and their use (65 episodes per dataset).

Group	Datasets	Use
v6 development	8	design, thresholds and all later tuning
v6 held-out	5	one-time v6 verdict (seen in v1–v5 as natural streams)
v7 confirmation	6	one-time drift-watch verdict
v8 confirmation	5	one-time harm-tier verdict
v10 confirmation	6	one-time test of stricter alerting rules
Natural streams	13	328 row-order windows, no injected shift (H5, C5)

4.3 Label-Free Signals

Table 4.2 lists the signals computed for every batch without labels. They fall into two families. *Input-shift* signals compare the batch's inputs with the reference; *model-behaviour* signals describe how the model reacts to the batch.

Table 4.2: Label-free signals computed per batch.

Signal	Family	Definition
Fused DEWS drift score	input shift	original prototype's weighted fusion of per-feature KS/PSI statistics, embedding MMD and a confidence component
Mean KS	input shift	mean two-sample KS statistic over features
Relevance-weighted shift	input shift	$\sum_j \text{importance}_j \cdot \text{KS}_j$, using f's permutation importance
Positive-rate shift	input shift	absolute change of the share of positive predictions from its burn-in mean
Confidence drop	behaviour	negative mean top-class probability of f
Entropy	behaviour	mean prediction entropy of f
Disagreement	behaviour	share of rows where f disagrees with a random forest [3] and a gradient-boosting model [6] trained on the same reference rows

4.4 The Two-Tier Monitor

Fig. 4.3 shows how one batch flows through both tiers and how delayed labels later confirm or clear an alert.

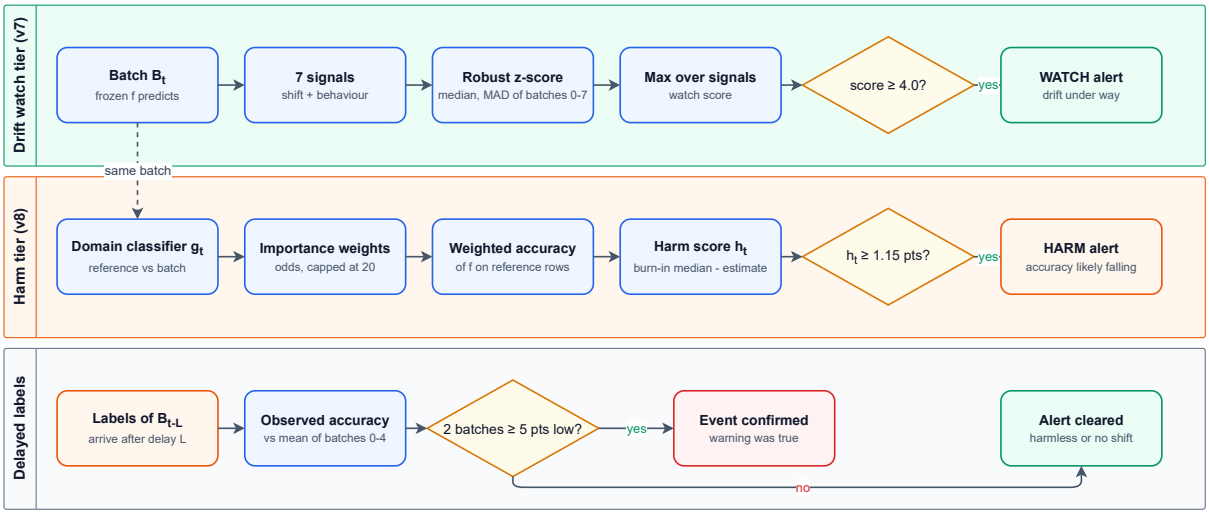


Figure 4.3: Per-batch data flow of the two-tier monitor and the confirmation by delayed labels.

4.4.1 Tier 1: drift watch (v7)

Raw drift statistics live in dataset-specific units: a KS statistic of 0.1 may be normal noise on one dataset and a large shift on another. The drift watch therefore expresses every signal relative to the stream's own burn-in. For a signal x_t,

$$z_t = \frac{x_t - \mathrm{med}(x_{0:8})}{\max\left(1.4826\,\mathrm{MAD}(x_{0:8}),\ 0.01\right)}, \tag{4.1}$$

a robust z-score in which the median and the median absolute deviation (scaled to match a standard deviation for normal data) are computed on batches 0–7. The drift-watch score is the maximum z_t over the seven signals of Table 4.2. A WATCH alert is raised when the score exceeds **4.011** standard deviations, a threshold chosen on development data. Because the score is measured in deviations from the stream's own normal, the same threshold is comparable across streams that start in a stable period.

4.4.2 Tier 2: harm tier (v8)

Whether a shift harms the model depends on whether it moves the data into regions where this particular model is wrong. The labeled reference set shows where that is. For each batch B_t, a logistic domain classifier g_t is trained to separate reference rows (label 0) from batch rows (label 1). For reference row x_i with correctness $c_i = \mathbb{1}[f(x_i) = y_i]$, the importance weight and the label-free accuracy estimate are

$$w_i = \min\left(\frac{\rho_i}{\bar{\rho}},\ 20\right), \quad \rho_i = \frac{g_t(x_i)}{1 - g_t(x_i)}, \qquad \hat{a}_t = \frac{\sum_i w_i c_i}{\sum_i w_i}, \tag{4.2}$$

where $\bar{\rho}$ is the mean odds over the reference. Under covariate shift, $\hat{a}_t$ targets the accuracy of f on B_t [24, 26]. The harm score is the estimated drop since the burn-in,

$$h_t = \mathrm{med}(\hat{a}_{0:8}) - \hat{a}_t, \tag{4.3}$$

in accuracy units. A HARM alert is raised when $h_t \geq 0.0115$ (an estimated drop of 1.15 accuracy points). The harm tier has no trained parameters beyond this threshold. Pure label changes are invisible to it by construction.

4.4.3 Threshold selection

Thresholds are chosen on development data only, with false alerts kept within 5 per 100 clean batches:

- **v6 rule** (also used by the drift watch): the threshold with the highest development T1 share; ties go to the higher threshold.
- **Separation rule** (harm tier): the threshold that maximises the T2 share on harmful shifts minus the alert share on harmless shifts.

4.5 Evaluation Protocol and Pre-Registration

Every experiment from v6 onward followed the workflow of Fig. 4.4. Before any confirmation data were obtained, a configuration module defining the datasets, seeds, method, threshold rule and each hypothesis with its pass rule was committed to git. Thresholds were chosen on development data and frozen in JSON files whose SHA-256 fingerprints were recorded in the protocol; the confirmation script refuses to run if a fingerprint does not match. Deviations, for example shuffling a dataset whose rows are stored sorted by class, were logged with date and reason before any affected episode was built. Each verdict was then computed once, by a script that applies the frozen pass rules.

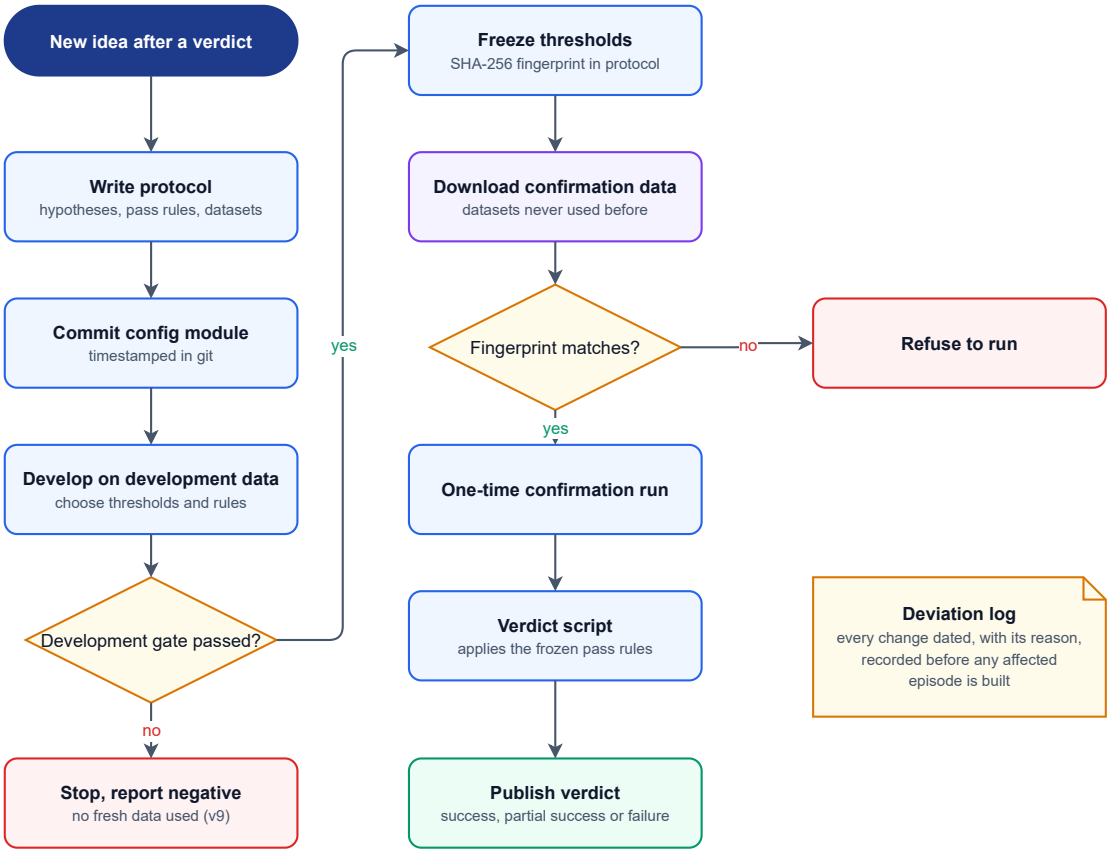


Figure 4.4: Pre-registration workflow used for every experiment from v6 onward.

Confidence intervals are 95% bootstrap intervals over episodes with 2000 resamples [4]; comparisons between two methods use a paired bootstrap

that resamples the same episodes for both.

4.6 Software Design

Fig. 4.5 shows the main parts of the code base and how they depend on each other. The core library `dews/` contains the benchmark, signal extraction, evaluation metrics, monitors and harm estimation. Experiment runners in `scripts/` use the library, read the frozen configurations and write all outputs to `artifacts/`. The dashboard, the paper figures and this report read only from `artifacts/`, so every number shown can be traced back to a saved result.

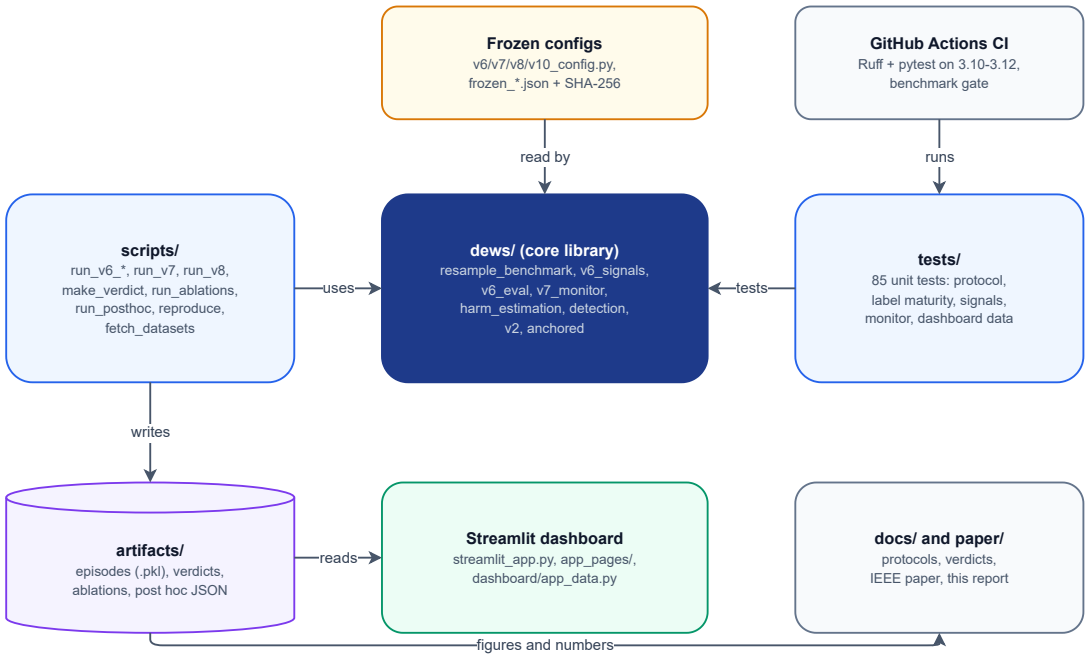


Figure 4.5: Software modules and their dependencies.

4.6.1 Dashboard design

The dashboard (Fig. 4.6) is a Streamlit [25] application with four pages. A plain Python module, `dashboard/app_data.py`, contains all data loading and replay logic without any Streamlit imports, so that it can be unit-tested. The *replay* function reproduces exactly what the monitor knew

at each batch: labels appear only after the selected delay, and scores are compared with the frozen thresholds.

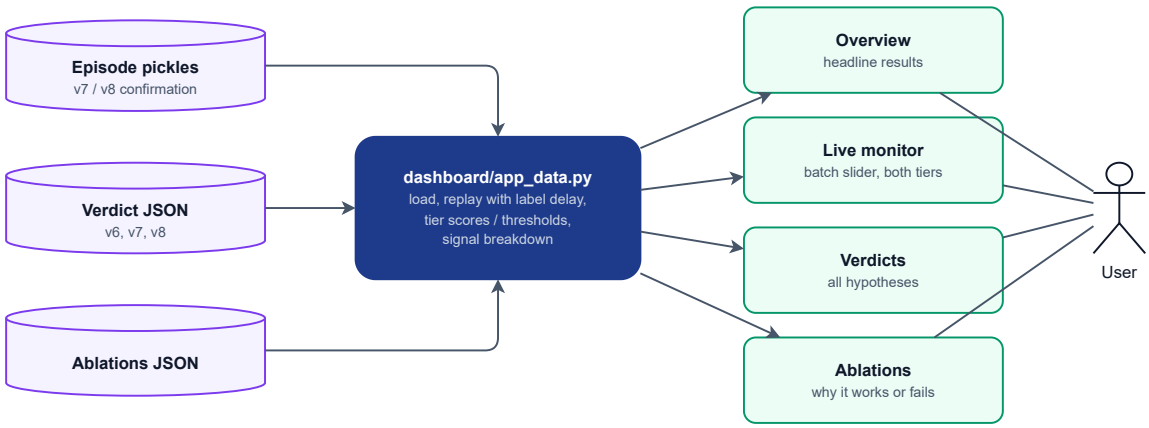


Figure 4.6: Data flow of the Streamlit dashboard.

Chapter 5

Implementation

5.1 Tools and Technologies

The system is written in Python. Models, auxiliary models and domain classifiers use scikit-learn [20]; statistics use NumPy and SciPy; tables use pandas; figures use matplotlib; and the dashboard uses Streamlit with Altair charts. Datasets are downloaded from OpenML [29]. The code is version-controlled with git and hosted on GitHub, where GitHub Actions runs linting, tests and a benchmark gate on every push. Diagrams were drawn with draw.io, and the paper and this report were typeset in LaTeX.

5.2 Development History: Ten Versions

The final design was reached in ten versions (Fig. 5.1). Versions v1–v5 were exploratory: they used synthetic scenarios and natural streams and were analysed after the fact. Versions v6–v10 were pre-registered (Section 4.5). Each version was motivated by a diagnosed failure of the previous one, and no earlier result was changed or withdrawn.

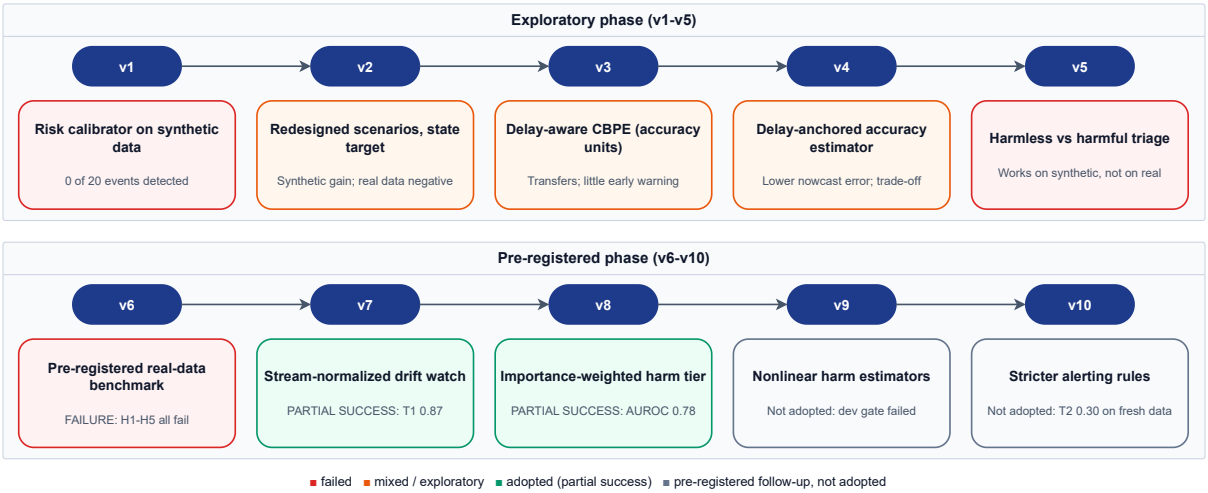


Figure 5.1: The ten versions of the system and what each showed.

5.2.1 v1: the original prototype's risk calibrator

The first evaluation used controlled synthetic episodes (clean, harmless shift, gradual and abrupt harmful shift) with seeds split into training, validation and test groups, and label delays of 0, 2 and 5 batches. On 120 held-out episodes the learned risk calibrator matched **0 of 20** harmful events at every delay, while the fixed DEWS drift score matched 9 of 20, all after the event had started. The diagnosis found two design errors rather than a weak idea: in all 50 seeds the accuracy event began on the same batch as the intervention, so no monitor could warn before it; and the risk target ("an event *starts* in the next 5 batches") labelled every post-onset batch negative, so the calibrator learned that more drift meant less risk.

5.2.2 v2: redesigned scenarios and a state target

v2 introduced noisy labels (baseline accuracy about 0.78), a randomised onset, relevant and nuisance features, a state target ("a degraded run starts or continues within the next 5 batches"), and a baseline that simply waits for delayed labels. On 250 held-out synthetic episodes the signals-only

calibrator warned before onset in 98% of harmful gradual episodes, about 4 batches ahead, where the fixed DEWS score gave no early warning (Table 5.1). A leave-one-dataset-out study on four natural streams (electricity, airlines, covertype, bank-marketing), however, was negative: drift signals ranked accuracy risk weakly, and every signal-based threshold that met the budget on the other datasets raised 4–16 times more false alerts on the held-out one.

Table 5.1: v2 controlled study, label delay $L = 2$, 250 held-out synthetic episodes.

Method	Recall	Early-warning rate	AUROC	Brier
Fixed DEWS score	0.00	0.00	0.56	–
Confidence only	0.44	0.27	0.50	–
Wait for delayed labels	1.00	0.42	0.90	–
Calibrator, signals only	0.73	0.42	0.83	0.161
Calibrator, signals + matured accuracy	1.00	0.56	0.94	0.086

5.2.3 v3: delay-aware confidence-based estimation

v3 expressed the score in accuracy units, which is what the event rule itself uses. It estimates each batch's accuracy as the mean top-class probability (as in CBPE) and corrects the bias with the residual of the latest 5 batches whose labels have already arrived. On four unseen natural streams this delay-aware CBPE stayed at 0.3–7 false alerts per 100 clean batches, against 10–11 for the drift scores, a first sign that units matter. It did not, however, give more early warning than waiting for labels.

5.2.4 v4: delay-anchored accuracy estimation

v4 anchored the estimate on the last matured accuracy and corrected it with a ridge regression on the change in 11 label-free signals, including

disagreement with auxiliary models trained on the same reference rows. Disagreement turned out to be the most informative label-free error signal (mean within-window Spearman correlation 0.41 with error, against 0.19 for confidence drop). The estimator reduced nowcast error compared with stale labels, increasingly with delay (Fig. 5.2), but against waiting for labels it was a trade-off rather than a win.

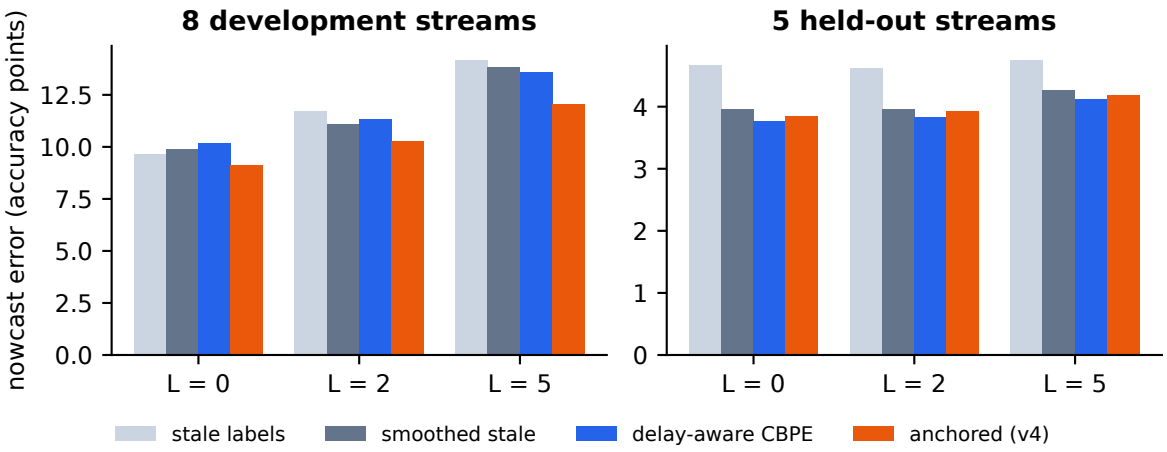


Figure 5.2: v4 accuracy nowcast error (mean absolute error in accuracy points) on 8 development streams (leave-one-dataset-out) and 5 held-out streams, by label delay.

5.2.5 v5: harmless versus harmful shifts

v5 asked whether the monitor could tell harmful from harmless shifts. A relevance-weighted shift, $\sum_j \text{importance}_j \cdot \text{KS}_j$, separated them perfectly on synthetic data (AUROC 1.0), where harmlessness meant "this feature does not matter". On 13 real streams it did not (AUROC 0.53 on held-out streams). A sustained-alert rule did not help either. These results showed that synthetic success does not imply real-data success, and motivated the pre-registered real-data benchmark of v6.

5.2.6 v6–v10: pre-registered experiments

v6 built the re-sampling benchmark of Section 4.2 and evaluated a delay-aware risk calibrator under a full pre-registered protocol. It failed, and its diagnosis (thresholds in raw units do not transfer) led to the stream-normalised drift watch of v7. v7's second tier failed to separate harm, which led to the reference-based harm tier of v8. v9 and v10 tried to improve the harm tier and were not adopted. The results are reported in Chapter 6.

5.3 Code Structure

Table 5.2 lists the main modules of the final system.

Table 5.2: Main modules of the code base.

Module	Responsibility
`dews/resample_benchmark.py`	builds benchmark episodes from real datasets (Algorithm 1)
`dews/v6_signals.py`	per-batch signal extraction with each episode's monitored model and auxiliary models
`dews/v6_eval.py`	event rule, warning tiers, false alerts, AUROC, threshold selection, v6 calibrator
`dews/v7_monitor.py`	robust normalisation and the drift-watch score
`dews/harm_estimation.py`	importance-weighted reference accuracy and the harm score
`dews/v6/v7/v8_config.py`	frozen protocols: datasets, seeds, hypotheses, pass rules
`dews/detection.py`, `monitoring.py`	original DEWS detectors (KS, PSI, JSD, MMD, confidence) and fused score
`dews/v2.py`, `anchored.py`	exploratory library (v2–v4), delay-anchored estimator
`scripts/run_v6_*.py`, `run_v7.py`, `run_v8.py`	development, freezing, download, extraction and one-time confirmation
`scripts/make_verdict.py`	applies the v6 pass rules and writes the verdict
`scripts/run_ablations.py`, `run_posthoc.py`	ablations and post hoc baselines
`scripts/reproduce.py`	one-command rebuild and verification of all verdicts
`streamlit_app.py`, `app_pages/`, `dashboard/`	the dashboard
`tests/`	unit tests

5.4 Key Implementation Details

5.4.1 Enforcing label delay

All label-derived features are computed from a helper that receives the current batch index and the delay and returns only the accuracies of batches $b \leq t - \max(L, 1)$. The same rule is used by the replay in the dashboard. Unit tests change the labels of batches that have not yet matured and check that no score at time t changes.

5.4.2 Per-batch monitoring

Algorithm 2 shows the per-batch logic of the final two-tier monitor.

Algorithm 2 Two-tier monitoring of one stream

Require: frozen f, reference (X_R, y_R), thresholds $\tau_W = 4.011$, $\tau_H = 0.0115$, delay L

$c_i \leftarrow \mathbb{1}[f(x_i) = y_i]$ for every reference row
for each batch B_t **do**
    compute the 7 signals $x_t^{(k)}$ from B_t, f and the auxiliary models
    fit domain classifier g_t on X_R (label 0) versus B_t (label 1)
    $w_i \leftarrow \min(\rho_i/\bar{\rho}, 20)$; $\hat{a}_t \leftarrow \sum_i w_i c_i / \sum_i w_i$
    if $t \geq 8$ **then**
        $z_t^{(k)} \leftarrow (x_t^{(k)} - \mathrm{med}(x_{0:8}^{(k)}))/\max(1.4826\,\mathrm{MAD}(x_{0:8}^{(k)}), 0.01)$
        if $\max_k z_t^{(k)} \geq \tau_W$ **then** raise WATCH
        if $\mathrm{med}(\hat{a}_{0:8}) - \hat{a}_t \geq \tau_H$ **then** raise HARM
    end if
    release labels of $B_{t-\max(L,1)}$; update observed accuracy; confirm or clear alerts by the event rule
end for

5.4.3 Pre-registration in code

Each pre-registered experiment has its own configuration module in `dews/` (for example `v7_config.py`; the v10 module is on its branch) that defines the datasets, seeds, settings and every hypothesis with its pass rule; a unit test checks the v6 configuration. Development scripts write the chosen thresholds to `artifacts/vN/frozen_vN.json`. The first 16 hexadecimal digits of the file's SHA-256 hash are recorded in the protocol document, and the confirmation script computes the hash again and refuses to run if it is not found there. Downloads of confirmation datasets are separate commands that were run only after this commit.

5.4.4 Parallel extraction

Signal extraction trains several models per batch and dominates the run time. Extraction runs as six independent shard processes, each writing its own result file, which are merged before evaluation. Rebuilding everything from scratch takes about two hours on the development laptop.

5.5 Dashboard

The dashboard has four pages. The *Overview* (Fig. 5.3) summarises the main results. The *Live monitor* (Fig. 5.4) replays a saved episode batch by batch: the user chooses the data group, dataset, episode type and label delay, and moves a slider over the batches. Key moments are shown as metrics (shift start, accuracy event, first drift-watch alert, first harm warning, label confirmation), with both tier scores divided by their thresholds, the accuracy seen through delayed labels, the label-free harm estimate, and a breakdown of which signals moved. The *Verdicts* page lists every pre-registered hypothesis with its result, and *Why it works* presents the ablation findings.

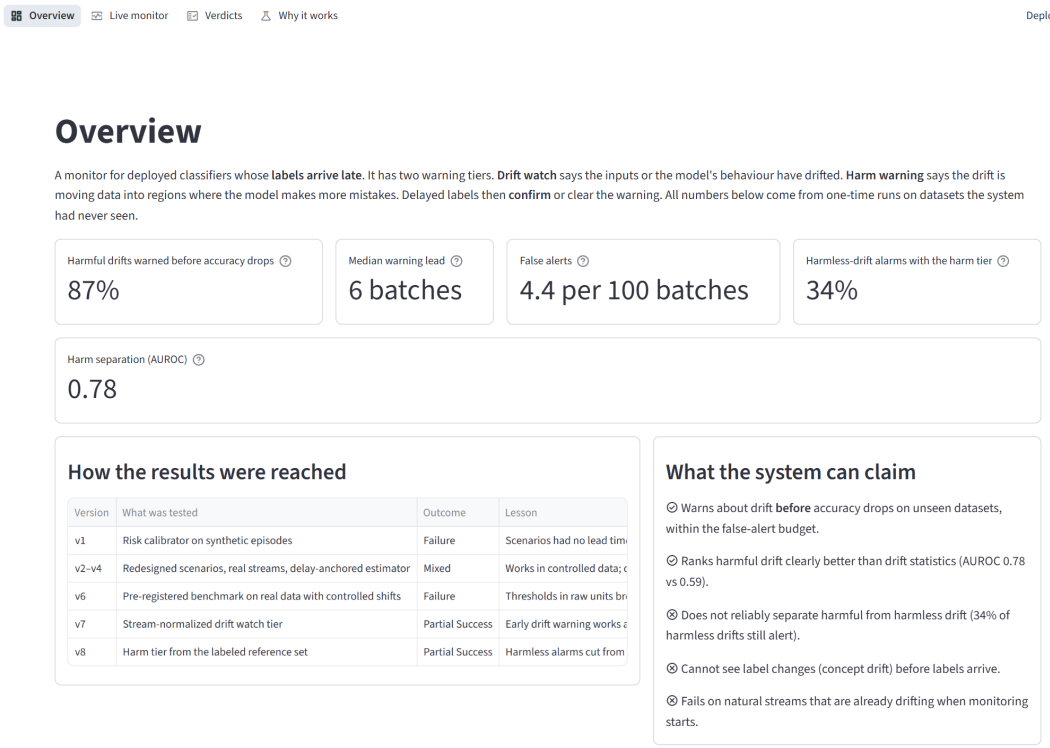


Figure 5.3: Dashboard: Overview page.

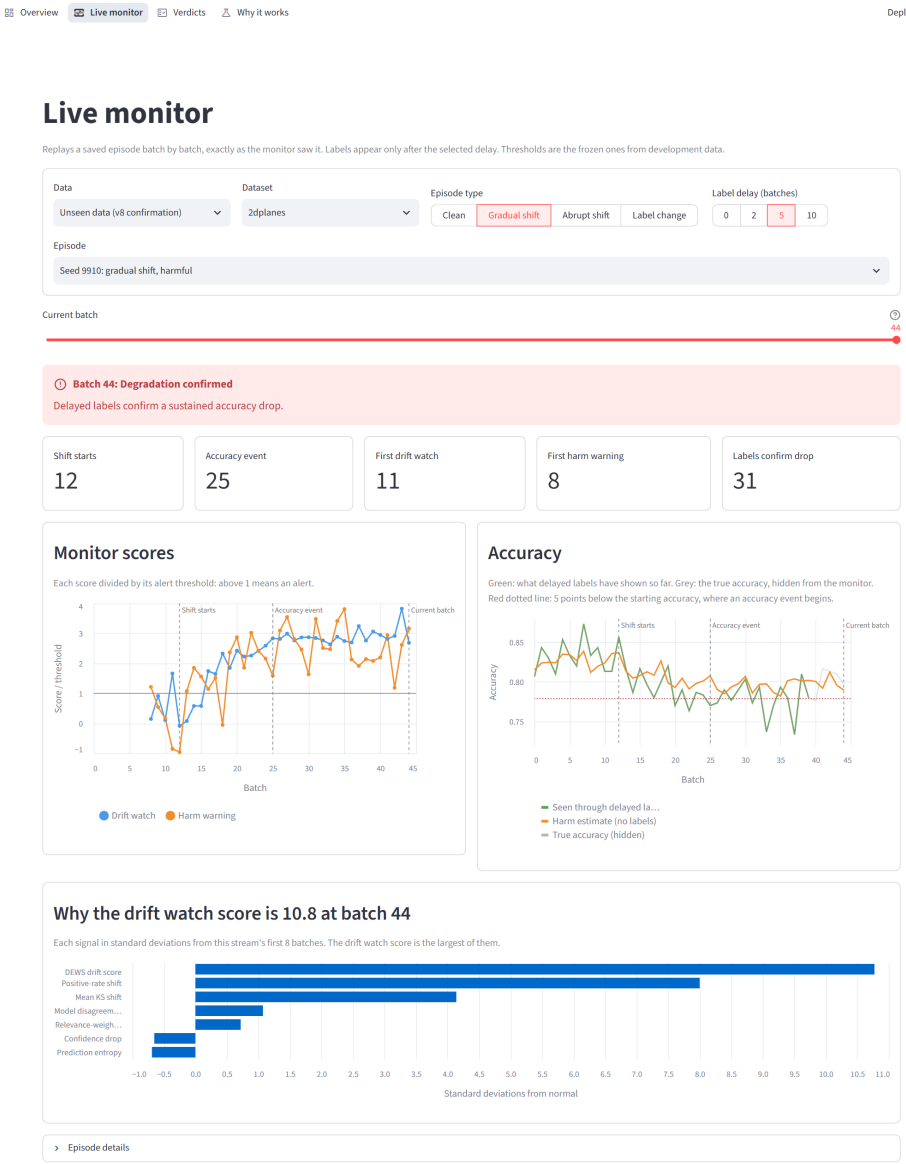


Figure 5.4: Dashboard: Live monitor replaying a harmful gradual shift on an unseen dataset (2dplanes, label delay 5). Alerts before the shift at batch 12 count as false alerts in the evaluation.

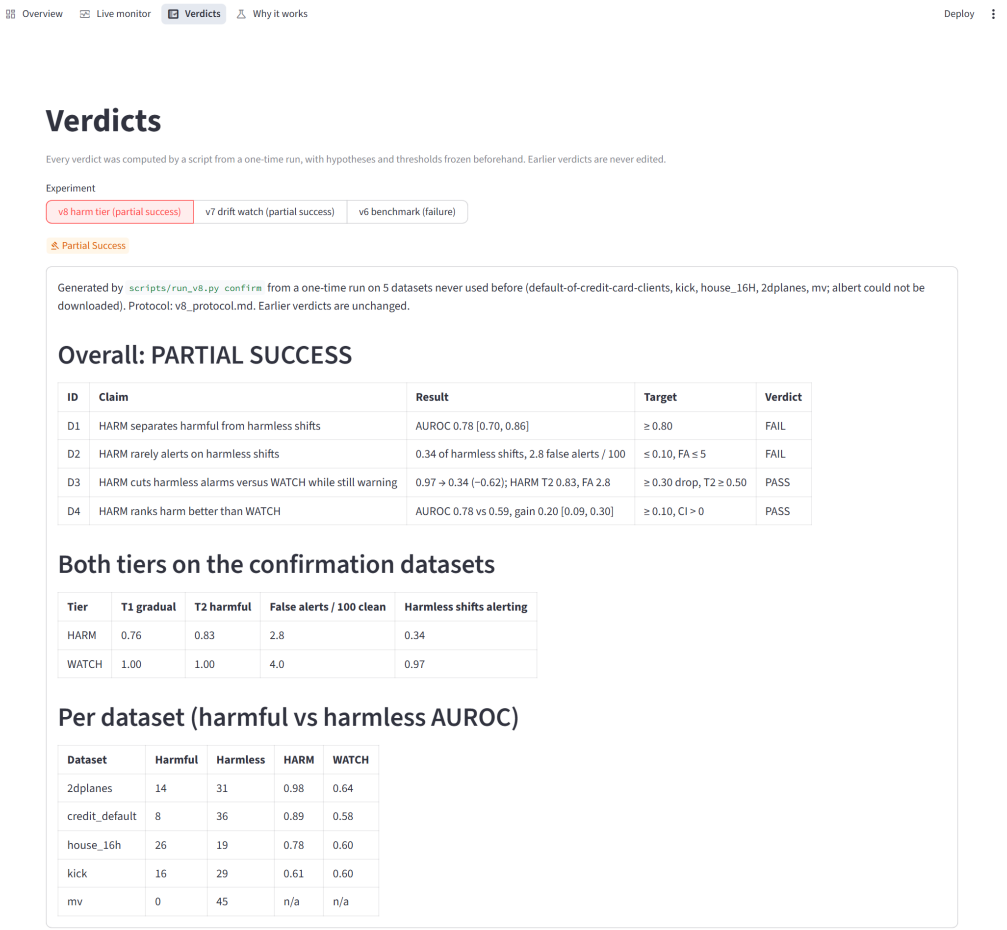


Figure 5.5: Dashboard: Verdicts page.

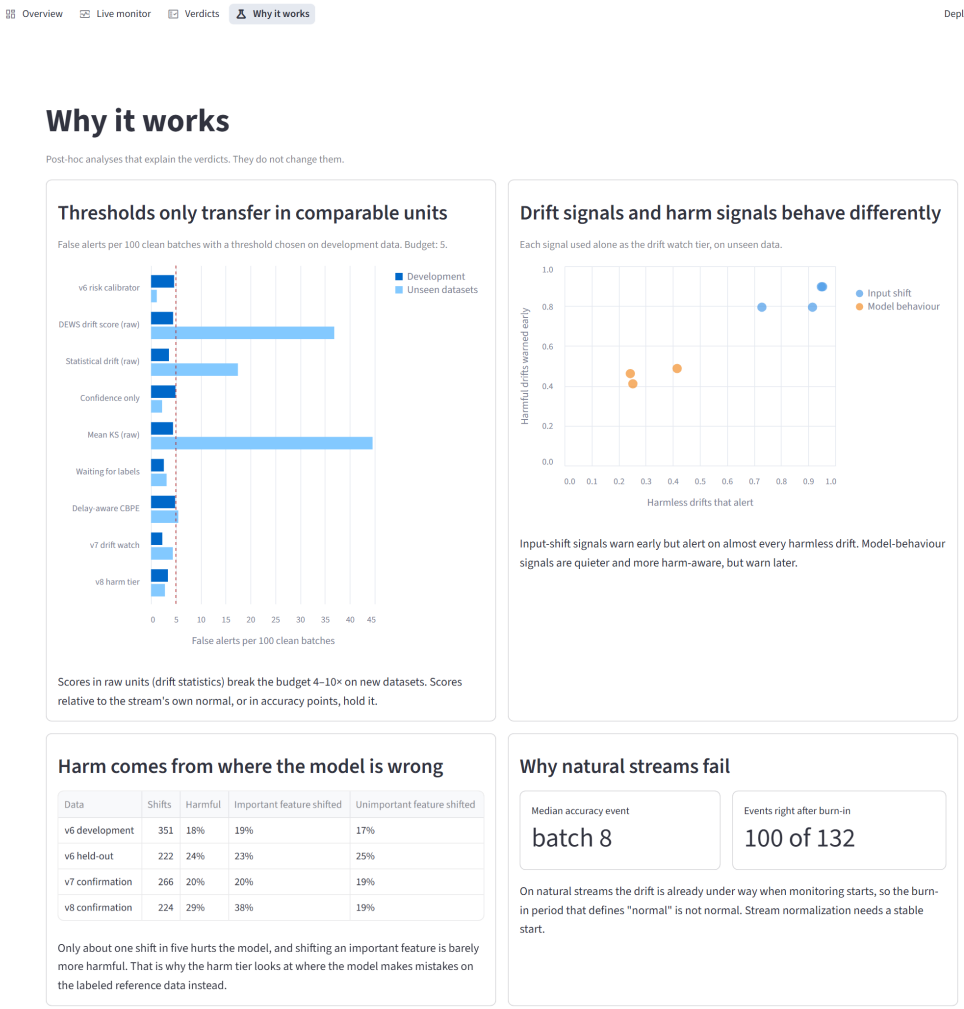


Figure 5.6: Dashboard: "Why it works" page with the ablation findings.

5.6 Testing and Continuous Integration

The repository contains 85 unit tests. They cover the protocol configuration, the event rule and warning tiers, label maturity (no score may change when unmatured labels change), the benchmark generator, signal extraction, the v7 monitor, the dashboard's data and replay logic, and components of the original prototype (detectors, alerts, forecasting, storage). The API tests of the legacy FastAPI service are skipped when that optional component is not installed.

GitHub Actions runs on every push and pull request (Fig. 5.7): it installs the core requirements, runs Ruff (pinned to version 0.17.4) and the full test

suite on Python 3.10, 3.11 and 3.12, and then runs a benchmark gate that checks the legacy detector's lag, alert count and accuracy. On the latest run of the main branch all jobs passed: 84 tests passed and 1 was skipped on each Python version.



Figure 5.7: Continuous-integration pipeline.

5.7 Reproducibility

Every episode is generated from a recorded seed, all models use fixed random states, and bootstrap intervals use fixed generator seeds. A lock file, `requirements-lock.txt`, records the exact library versions used for the frozen verdicts. The command `python scripts/reproduce.py` rebuilds whatever is missing and fails if any verdict differs from the saved one. In a clean environment, ten regenerated v8 episodes matched the saved ones to within 4×10^{-11}, and the v6, v7 and v8 verdicts were reproduced exactly. The v9 and v10 code is kept on separate branches of the repository because neither was adopted.

5.8 Cleaning Up the Prototype

The original repository contained a Kafka, Celery, Redis and Grafana stack and claims of production readiness that had never been evaluated. Because this infrastructure did not contribute to answering the research question, it was deleted, and the README was rewritten around the evaluated results. The legacy FastAPI service, MLflow tracking and PostgreSQL storage were

kept as optional components in a separate requirements file.

Chapter 6

Results and Discussion

All results in this chapter use logistic regression as the monitored model and a label delay of $L = 5$ batches with all labels, unless stated otherwise. Intervals are 95% bootstrap intervals over episodes. Every number comes from a saved verdict or analysis file in `artifacts/`; the post hoc analyses are marked as such and do not change any verdict.

6.1 v6: The Original Design Fails

6.1.1 Benchmark gate

The v6 protocol required that at least 25% of shift episodes on development data be harmful, so that the benchmark would contain enough harm to measure. The gate failed at every allowed shift strength (Fig. 6.1): only 17–19% of shifts were harmful, and raising the strength from 1.5 to 4.0 did not change this. Shifts of the more important features were no more often harmful than shifts of the less important ones (0.18 versus 0.17). Because no allowed strength passed and forcing harm (for example, shifting towards misclassified rows) would have made the benchmark partly circular, we proceeded at $\lambda_{max} = 1.5$ by a decision recorded before any held-out data were generated. This is reported as a deviation.

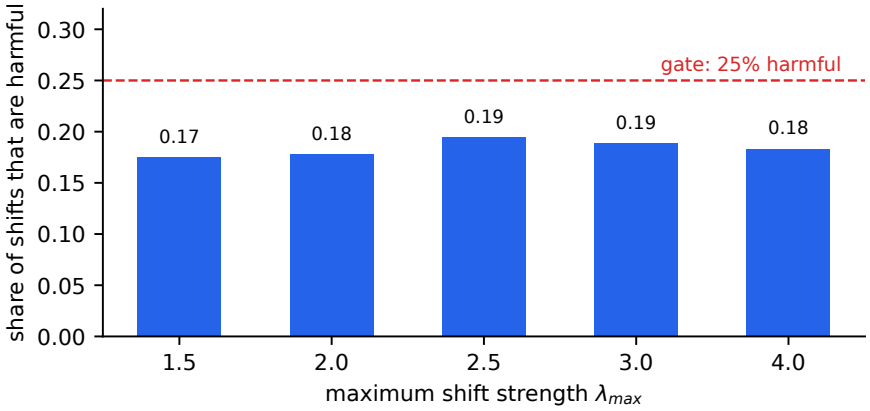


Figure 6.1: Share of development shift episodes that harmed the model, by maximum shift strength. The pre-registered gate required at least 25%.

6.1.2 Pre-registered verdict

Table 6.1 shows the verdict on the 5 held-out datasets (53 harmful and 169 harmless shifts). All five hypotheses failed. The learned calibrator warned before the drop on only 22% of harmful gradual shifts and did not beat waiting for delayed labels. The original DEWS score's higher warning rates on some datasets (Fig. 6.2) came from alerting almost constantly: on held-out data it raised 36.9 false alerts per 100 clean batches against a budget of 5, while the calibrator raised only 1.2.

Table 6.1: v6 pre-registered hypotheses on the held-out datasets. Overall verdict: FAILURE.

Hypothesis (target)	Result	Verdict
H1: T1 on harmful gradual shifts ($\geq$0.60, lead $\geq$2, FA $\leq$5)	0.22 [0.08, 0.36], lead 12.5, FA 1.2	Fail
H2: T2 gain over waiting for labels ($\geq$0.10, CI >0)	0.83 vs 0.85, -0.02 [-0.11, 0.08]	Fail
H3: harm AUROC ($\geq$0.80); harmless alarms ($\leq$0.10)	0.59 [0.51, 0.68]; 0.37	Fail
H4: beats the DEWS score ($\geq$4 of 5 datasets)	2 of 5	Fail
H5: natural streams, T2 gain over labels ($\geq$0)	-0.01 [-0.08, 0.06]	Fail

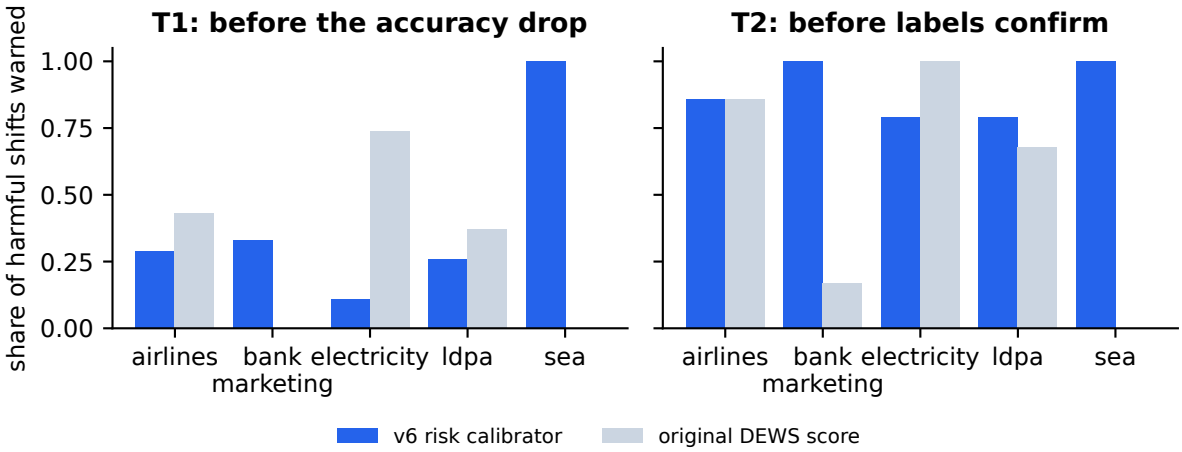


Figure 6.2: v6 per held-out dataset: share of harmful shifts warned before the drop (T1) and before labels confirm (T2).

6.1.3 Secondary settings

With only 20% of rows ever labeled (Table 6.2), the gradient-boosting system warned before confirmation more often than the delayed-label trend, but its false alerts exceeded the budget at most delays, and the logistic system stayed below the label trend. We therefore make no claim from these settings.

Table 6.2: v6 secondary settings with 20% of labels (held-out): T2 on harmful shifts and false alerts per 100 clean batches.

Model, delay	System T2	Labels T2	System FA	Labels FA
Logistic, $L = 0$	0.43	0.58	1.7	4.5
Logistic, $L = 2$	0.38	0.57	1.0	4.0
Logistic, $L = 5$	0.49	0.70	1.0	5.0
Logistic, $L = 10$	0.55	0.81	1.1	5.7
Gradient boosting, $L = 0$	0.64	0.48	3.9	4.0
Gradient boosting, $L = 2$	0.73	0.57	5.7	5.1
Gradient boosting, $L = 5$	0.73	0.61	5.3	4.9
Gradient boosting, $L = 10$	0.82	0.80	7.7	5.9

6.2 Why v6 Failed: Thresholds in Raw Units Do Not Transfer

Fig. 6.3 compares, for each method, the false-alert rate on development data (where its threshold was chosen) with the rate on data never used for threshold selection. Scores in dataset-specific units broke the budget by a factor of 3.5–9 (fused DEWS score 4.4 → 36.9; mean KS 4.4 → 44.6), while the learned calibrator under-alerted (4.6 → 1.2). Scores expressed relative to the stream's own normal (drift watch, 2.3 → 4.4) or in accuracy points (harm tier, 3.4 → 2.8) stayed within the budget on their confirmation data. An ablation of the v6 calibrator confirmed the same mechanism inside the model: removing the raw disagreement levels, which differ widely across datasets, raised its T1 from 0.22 to 0.42. This did not hold everywhere, however: the drift watch raised 8.0 false alerts per 100 clean batches on natural streams, and the harm tier raised 12.7 on the six v10 datasets (Section 6.7).

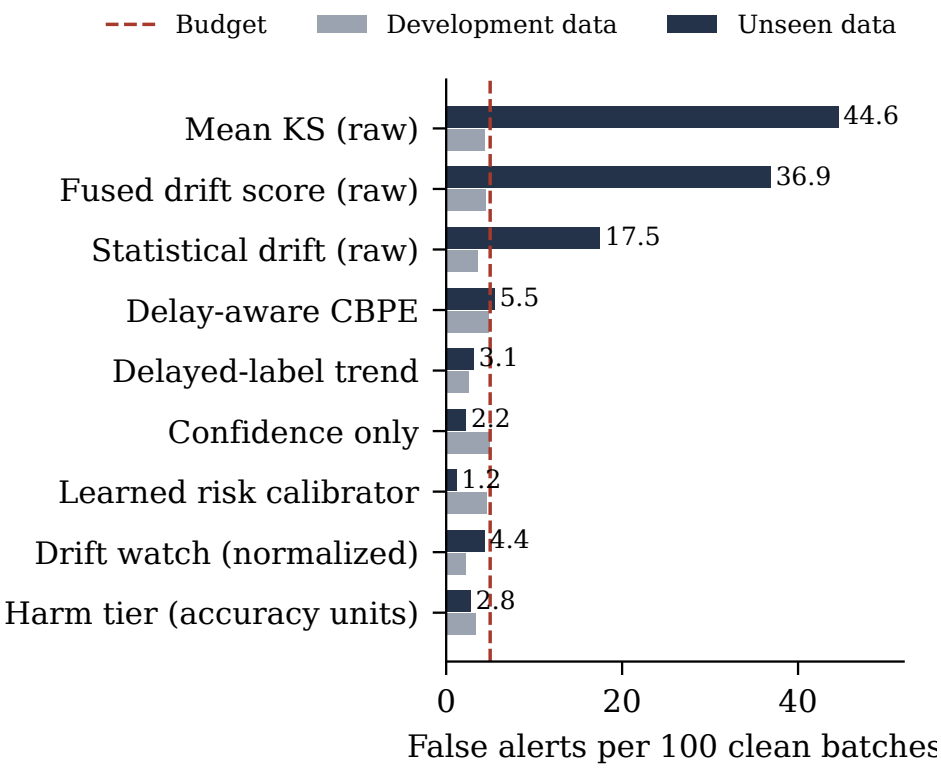


Figure 6.3: False alerts per 100 clean batches with thresholds chosen on development data, on development and on unseen data (v6 held-out data for the v6 methods; v7 and v8 confirmation data for the drift watch and the harm tier). Dashed line: budget of 5.

6.3 v7: The Drift-Watch Tier

On six fresh datasets (39 harmful gradual shifts) the drift watch warned before the accuracy drop on **0.87** [0.77, 0.97] of harmful gradual shifts, a median of **6 batches** ahead, with **4.4** false alerts per 100 clean batches, and beat the fused DEWS score on 5 of 6 datasets (Table 6.3, Fig. 6.4, Fig. 6.5). Its pre-confirmation warning rate (1.00 against 0.96 for waiting for labels) did not reach the pre-registered gain of 0.10. It alerted on 96% of harmless shifts: it detects drift but cannot tell whether the drift is harmful.

v7 also pre-registered a second tier, a logistic calibrator over the normalised signals. It failed its separation hypothesis (AUROC 0.59, alerts on 53% of harmless shifts), which motivated the reference-based harm tier of

v8.

Table 6.3: v7 pre-registered hypotheses on six fresh datasets. Overall verdict: PARTIAL SUCCESS.

Hypothesis (target)	Result	Verdict
C1: WATCH T1 ($\geq$0.60, lead $\geq$2, FA $\leq$5)	0.87 [0.77, 0.97], lead 6, FA 4.4	Pass
C2: WATCH T2 gain over labels ($\geq$0.10, CI >0)	1.00 vs 0.96, 0.04 [0.00, 0.10]	Fail
C3: WARNING separates harm (AUROC $\geq$0.80, harmless $\leq$0.10)	AUROC 0.59; harmless 0.53	Fail
C4: WATCH beats the DEWS score ($\geq$70% of datasets)	5 of 6	Pass
C5: natural streams, T2 gain over labels ($\geq$0)	0.44 vs 0.87, -0.43 [-0.53, -0.32]	Fail

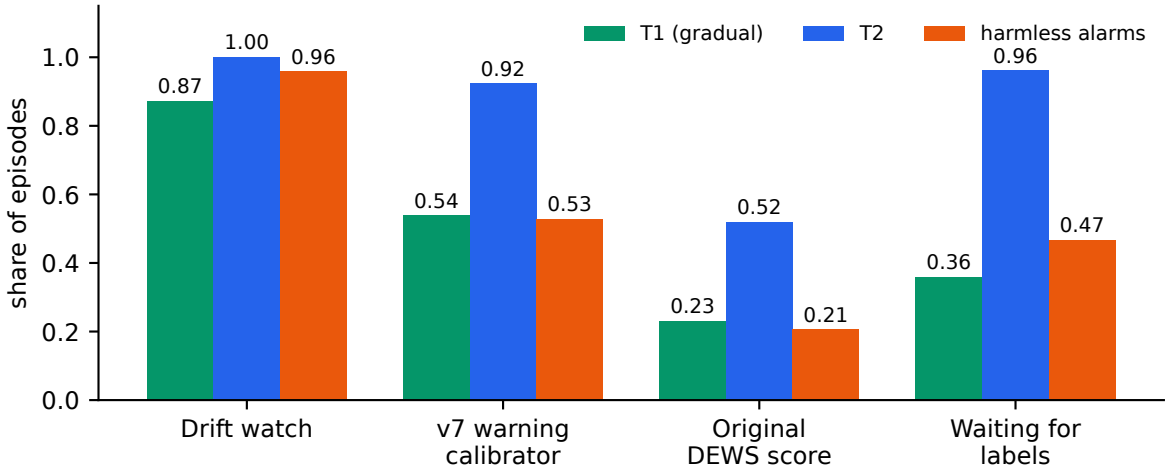


Figure 6.4: v7 confirmation data: early warning (T1), pre-confirmation warning (T2) and harmless-shift alarms of each method.

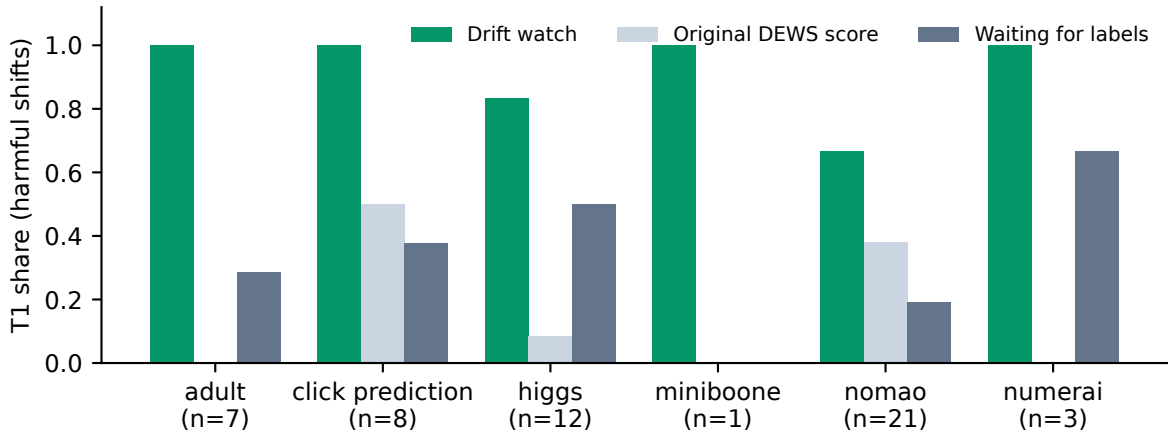


Figure 6.5: v7 per confirmation dataset: T1 share of harmful shifts (n = number of harmful shifts).

6.3.1 Natural streams

On the 328 natural-stream windows the drift watch failed (T2 0.44 against 0.87 for the label trend) and exceeded the budget (8.0 false alerts per 100 clean batches against 6.3 for the label trend). The reason is visible in the data: the median degradation began at batch 8, the end of the burn-in; 100 of the 132 events began within 5 batches after the burn-in; and accuracy varied by about 12% within the burn-in itself. The "normal" that the drift watch measures against was therefore not normal. Stream normalisation needs a genuinely stable period after deployment.

6.4 v8: The Harm Tier

On five further fresh datasets (64 harmful and 160 harmless shifts) the harm tier cut alarms on harmless shifts from **0.97** (drift watch) to **0.34**, while still warning before labels could confirm on 0.83 of harmful shifts and before the drop on 0.76 of harmful gradual shifts, at 2.8 false alerts per 100 clean batches (Table 6.4). It separated harmful from harmless shifts

with an AUROC of **0.78** [0.70, 0.86], against 0.59 for the drift watch (gain 0.20 [0.09, 0.30]). It passed two of four hypotheses and missed the AUROC target of 0.80 and the harmless-alarm target of 0.10.

Table 6.4: v8 pre-registered hypotheses on five fresh datasets. Overall verdict: PARTIAL SUCCESS.

Hypothesis (target)	Result	Verdict
D1: HARM separates harm (AUROC ≥0.80)	0.78 [0.70, 0.86]	Fail
D2: HARM rarely alerts on harmless shifts (≤0.10, FA ≤5)	0.34, FA 2.8	Fail
D3: cuts harmless alarms vs WATCH (≥0.30) with T2 ≥0.50	0.97 → 0.34; T2 0.83	Pass
D4: ranks harm better than WATCH (gain ≥0.10, CI >0)	0.78 vs 0.59, gain 0.20 [0.09, 0.30]	Pass

Per dataset (Fig. 6.6) the harm tier separated harm well on three of the four datasets that produced harmful shifts and was near chance on *kick*; *mv* produced no harmful shift at all.

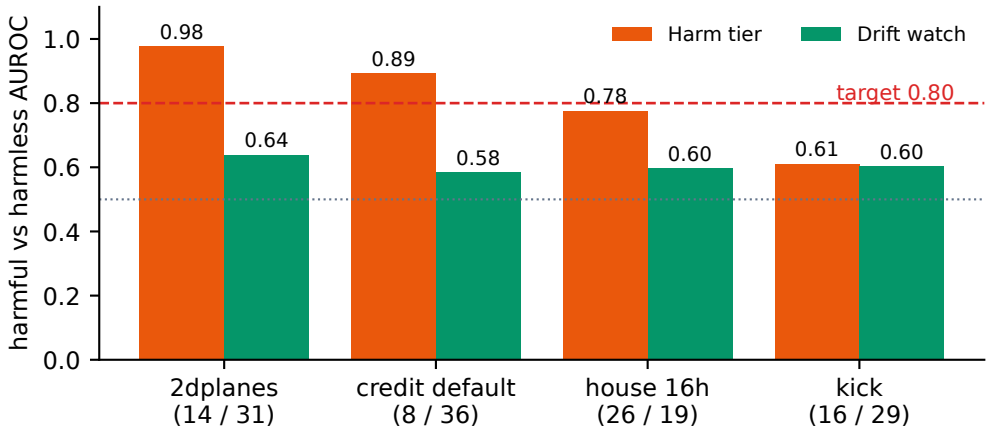


Figure 6.6: v8 harmful-versus-harmless AUROC per confirmation dataset (number of harmful / harmless shifts in brackets).

Fig. 6.7 shows one harmful gradual shift on an unseen dataset. The harm tier first alerts at batch 16, 14 batches before the accuracy event and

20 before delayed labels could confirm it.

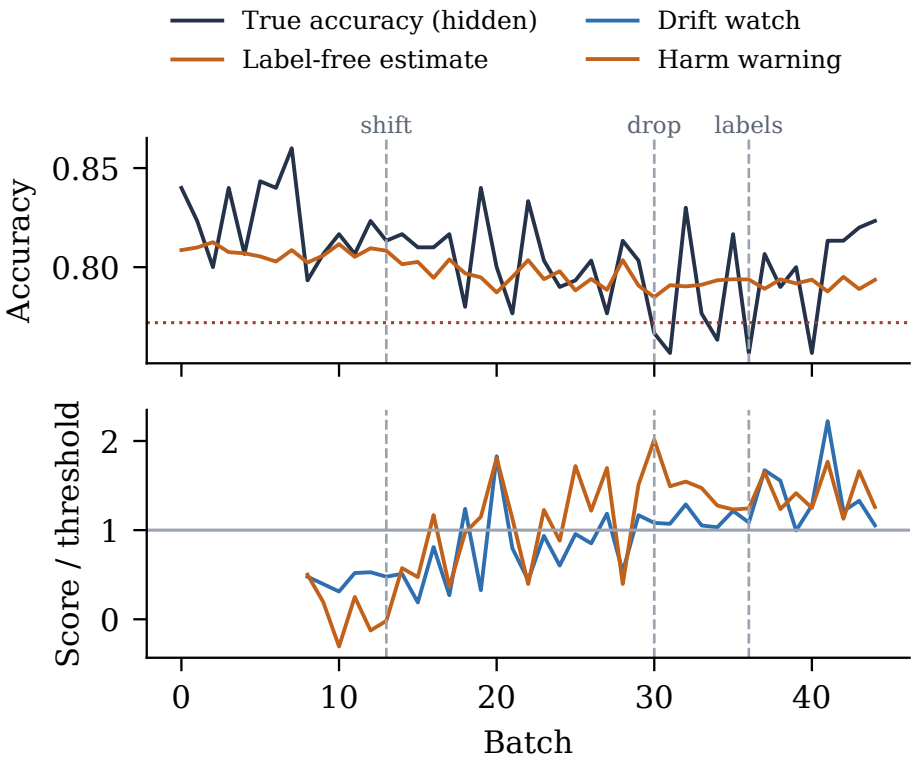


Figure 6.7: A harmful gradual shift on an unseen dataset (credit default, $L = 5$). Top: true accuracy (hidden from the monitor) and the label-free estimate; dotted line: 5 points below baseline. Bottom: tier scores divided by their thresholds; above 1 is an alert.

6.5 Same-Data Baselines (Post Hoc)

To check whether the harm tier is better than simpler label-free estimates, we scored other signals on the v8 confirmation data in exactly the same way (Fig. 6.8). Confidence drop, entropy and delay-aware CBPE reached only 0.56–0.59, and the paired bootstrap interval of the harm tier's advantage over each of them excluded zero. kNN reference accuracy (0.76) and disagreement with models trained on the reference rows (0.73), the other two methods that learn from the labeled reference, were not distinguishable from the harm tier. Confidence drop had reached 0.77 on the v7 data (Table 6.5), so confidence alone is not stable across datasets.

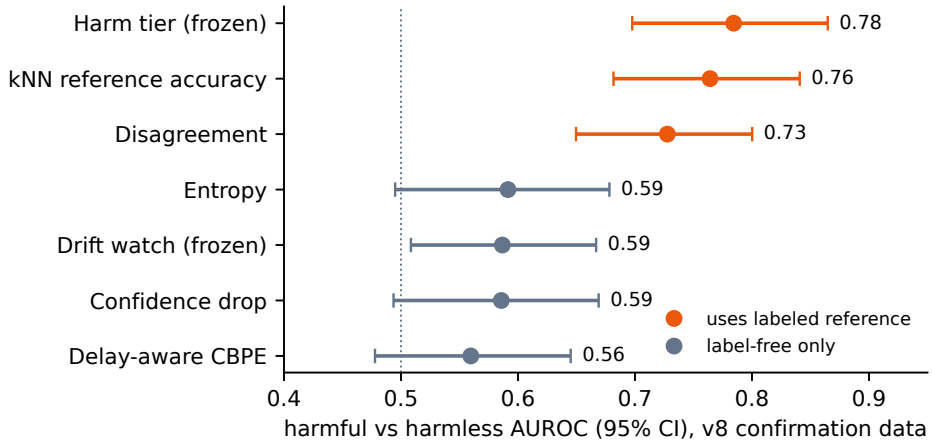


Figure 6.8: Harmful-versus-harmless AUROC of label-free signals on the v8 confirmation data (post hoc), with 95% bootstrap intervals.

6.6 Why Harmless Shifts Still Alert

In a post hoc analysis of the v8 confirmation data, streams with no shift at all raised a harm alert in 40% of episodes and harmless shifts in 34%, against 88% for harmful shifts. The harm tier therefore treats harmless shifts like no shift: the remaining alarms are background false alerts that accumulate over about 33 monitored batches. With a per-batch rate of 2.8 per 100, the chance of at least one alert in 33 batches is $1 - 0.972^{33} \approx 0.6$. Our 10% target for harmless alarms was inconsistent with our own per-batch budget. We set this target before noticing the inconsistency and leave the verdict unchanged.

Fig. 6.9 shows the full trade-off on the same data. Over most of the range of harmless alarms the harm tier warns on far more harmful shifts than the drift watch. On this data a stricter harm threshold would appear better than the frozen one, but when that idea was frozen and tested on six other datasets (v10, square marker) it warned on only 30% of harmful shifts, so choosing a threshold from this curve after seeing the results would

have been misleading.

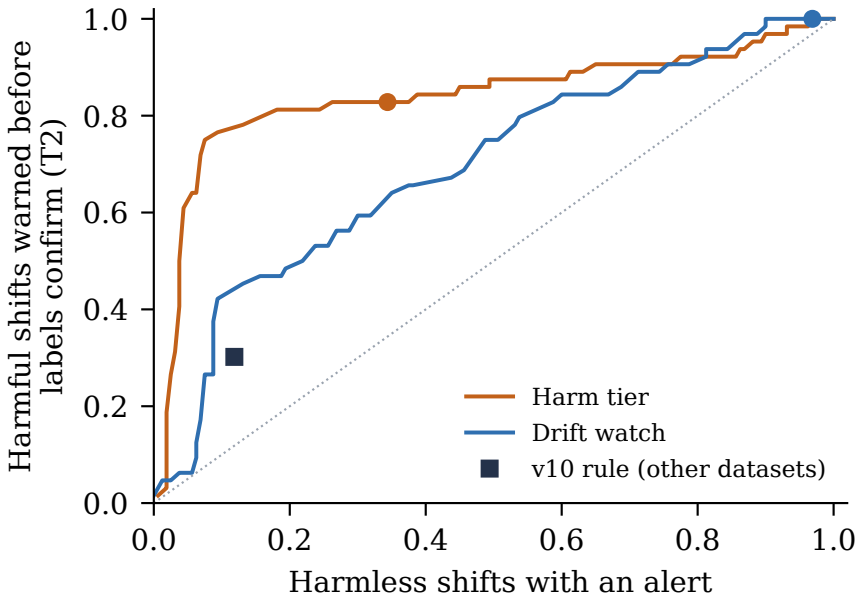


Figure 6.9: Post hoc operating curves on the v8 confirmation data; each point is a threshold. Circles: frozen thresholds. Square: the stricter v10 rule, frozen on development data and evaluated on six different datasets.

6.7 Pre-Registered Follow-Ups That Failed

v9: nonlinear harm estimators. A gradient-boosting domain classifier and an error predictor trained on out-of-fold reference correctness both separated harm worse than the linear estimator on development data (AUROC 0.70 against 0.77). In a post hoc check, no ensemble of these, the linear estimator, kNN accuracy and confidence (averaging within-dataset ranks) exceeded 0.76, against 0.75 for the linear estimator scored the same way. The development gate of 0.80 failed, and no fresh data were used.

v10: stricter alerting rules. Stricter thresholds, exponential smoothing, rolling means, CUSUM, stream-normalised harm scores and gating by the drift watch all reduced harmless alarms at the cost of early warnings. The best rule (a z-scored harm score) passed the development gate, but

on six fresh datasets it warned before labels on only 0.30 of harmful shifts against a required 0.50, while reducing harmless alarms to 0.12. Neither v9 nor v10 was adopted.

On the same six v10 datasets the frozen v8 rule warned before labels on 0.74 of harmful shifts but alerted on 58% of harmless shifts and raised **12.7** false alerts per 100 clean batches, more than twice the budget. Its calibration therefore did not hold on every new dataset. Fig. 6.10 places these operating points side by side.

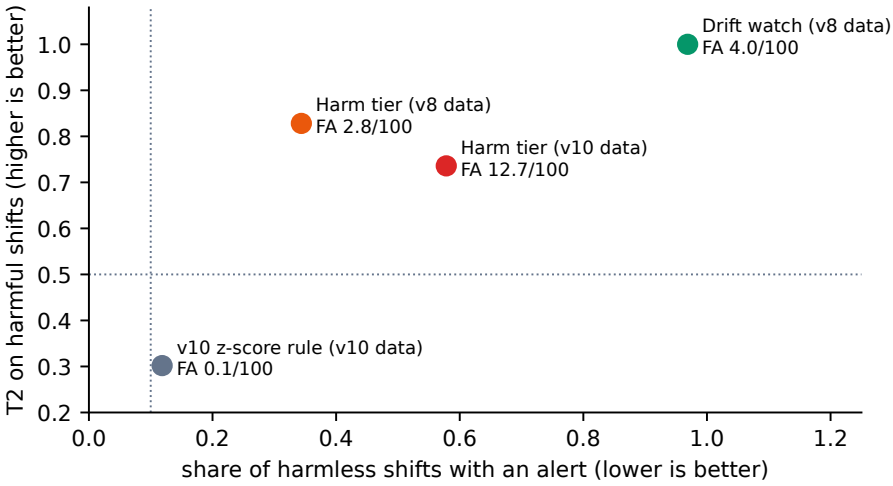


Figure 6.10: Harmless-shift alarms against pre-confirmation warnings for the frozen tiers and the v10 rule; FA = false alerts per 100 clean batches.

6.8 Ablations

6.8.1 Two families of signals

Using each normalised signal alone as the drift watch on the v7 confirmation data (Table 6.5, Fig. 6.11) shows two families with opposite strengths. Input-shift signals warned early on 79–90% of harmful gradual shifts but could not tell harm from harmlessness (AUROC about 0.5). Model-behaviour signals were harm-aware (AUROC 0.72–0.77) but warned later. Removing any single signal from the drift watch changed little, so

no signal is a single point of failure.

Table 6.5: Each normalised signal used alone (v7 confirmation data, post hoc).

Family	Signal	T1	Harmless alarms	AUROC
Input shift	fused DEWS score	0.90	0.95	0.52
	mean KS	0.90	0.95	0.53
	relevance-weighted shift	0.79	0.92	0.51
	positive-rate shift	0.79	0.73	0.54
Model behaviour	disagreement	0.49	0.42	0.72
	confidence drop	0.46	0.24	0.77
	entropy	0.41	0.25	0.75
All 7 (drift watch)		0.87	0.96	0.51

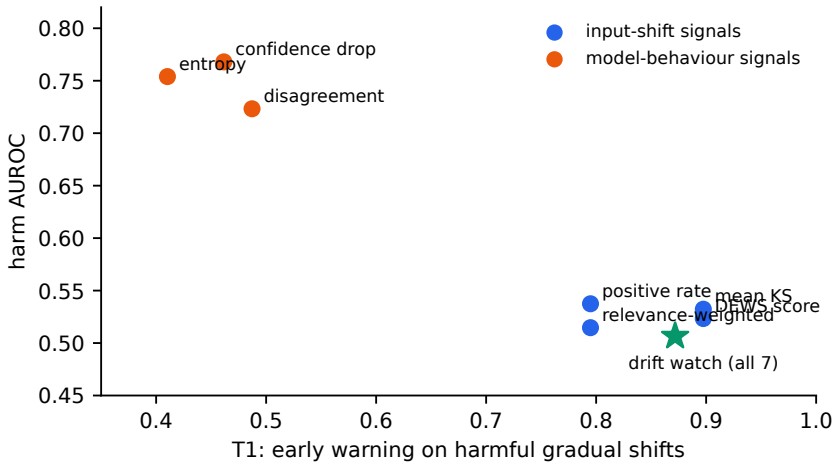


Figure 6.11: Early warning versus harm separation of each signal used alone (v7 confirmation data).

6.8.2 Which harm estimator

All reference-based estimators reached about the same separation on the v8 confirmation data (AUROC 0.76–0.78), far above every drift signal (Table 6.6). The gain comes from using the labeled reference, not from the particular estimator. The variants trade sensitivity for quietness: nearest-neighbour accuracy is the quietest but warns less.

Table 6.6: Harm-estimator variants on the v8 confirmation data (thresholds by the separation rule).

Estimator	T1	T2	FA	Harmless alarms	AUROC
Importance-weighted (frozen)	0.76	0.83	2.8	0.34	0.78
Nearest-neighbour	0.49	0.70	0.2	0.17	0.76
Average of both	0.71	0.83	1.0	0.29	0.78

6.8.3 Why most shifts are harmless

In every data group about 70–80% of shifts did not harm the model (Fig. 6.12). Whether harm occurred was barely related to the importance of the shifted feature (the difference appeared only in the v8 group) and not related to the model's baseline accuracy. Harm depends on whether the shift moves data into regions where this particular model is wrong. That explains why feature-importance weighting (v5) failed and why reference-based estimation (v8) was the first approach to separate harmful from harmless shifts.

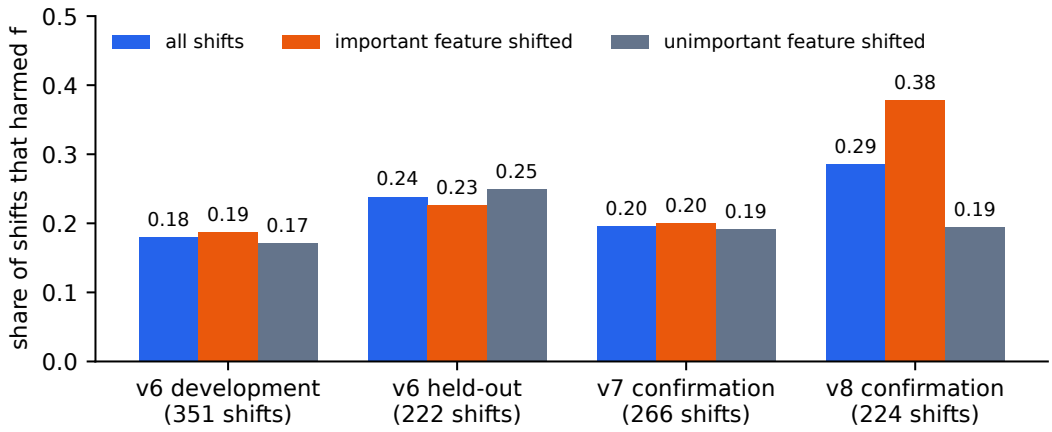


Figure 6.12: Share of shift episodes that harmed the model, by data group and by importance of the shifted feature.

6.9 Discussion

6.9.1 Lessons

1. **Normalisation made thresholds transfer, but is not sufficient.** The decisive change between the failed and the partially successful designs was expressing signals in units comparable across streams, not a better model. Even so, the harm rule exceeded the budget on the v10 datasets and the drift watch on natural streams. Alert thresholds tuned on historical data should be validated on data from other sources before deployment.

2. **Separate a sensitive drift tier from a specific harm tier.** Input-shift signals are good for early notice; model-behaviour and reference-based signals are needed to judge harm.

3. **Keep a labeled reference sample.** In our data, only methods that used it separated harm consistently across datasets.

4. **Make per-episode targets consistent with per-batch budgets.** A 2.8% per-batch false-alert rate already implies about a 60% chance of an alarm over 33 batches.

5. **Report negative results.** Several attractive ideas (online updating in v2, relevance weighting in v5, nonlinear estimators in v9, stricter rules in v10) did not survive testing on new data; recording them prevents the same mistakes being repeated.

6.9.2 Threats to validity

The v7 and v8 designs were motivated by the v6 failure, so they are informed by data we had already seen; we mitigated this by confirming each

on datasets downloaded only after its design was frozen, but the sequence of experiments was not planned in advance. The 13 v6 datasets had been explored in v1–v5, so the v6 held-out results are not fully out of sample. Harmful-shift counts per confirmation group (52–64; 39 gradual in v7) give confidence intervals of roughly ± 0.1. The event rule (two batches at least 5 points below baseline) and the horizon are conventional choices that we did not vary. Pre-registration was internal (git commits), not on a public registry.

6.9.3 Limitations

Shifts are constructed by re-sampling real rows, which preserves $P(y \mid x)$ and thus favours covariate-shift methods; changes in the label relationship are invisible to every label-free signal until labels arrive. The confirmed model is logistic regression only; the burn-in assumes a stable start, which failed on natural streams; the setting is binary tabular data with 20 features and batches of 300 rows; and the per-dataset numbers of harmful shifts are small, so intervals are wide. The harm tier trains a domain classifier on every batch, and its run-time cost was not measured systematically.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project set out to build an ML model drift early-warning system that warns about degradation, not merely drift, before delayed labels can confirm it. We built a benchmark that re-samples real tabular data so that whether a shift harms the model is measured from true labels, and we evaluated ten versions of the system, five of them with pre-registered hypotheses, frozen thresholds and one-time runs on unseen datasets.

The results are mixed, and we report them as they are:

- The original design, a learned risk calibrator over raw drift scores, failed all its hypotheses, because thresholds in dataset-specific units did not transfer across datasets.
- A stream-normalised **drift watch** gave early warning of harmful drift within its false-alert budget on benchmark streams: on six fresh datasets it warned before the accuracy drop on 87% of harmful gradual shifts, a median of six batches ahead. It cannot tell harmful from harmless drift, and it failed on natural streams where drift had begun before monitoring.

- An importance-weighted **harm tier** reduced alarms on harmless shifts from 97% to 34% and separated harmful from harmless shifts with an AUROC of 0.78, short of the 0.80 target but above confidence-based baselines on the same data. On a later set of datasets its false-alert rate exceeded the budget.
- Nonlinear estimators, ensembles and stricter alerting rules did not improve on it.

Without labels, our best estimator separated harmful from harmless shifts only partly, and it needed a labeled reference set. The practical lessons are to express alert scores in units that are comparable across streams, to validate thresholds on data from other sources, to separate a sensitive drift tier from a specific harm tier, and to keep a labeled reference sample.

Beyond the research findings, the project delivers a working and reproducible product: the monitoring library, frozen configurations and verdict scripts, a dashboard that replays monitored streams with delayed labels, 85 unit tests with continuous integration, and a research paper.

7.2 Future Work

1. **Better burn-in handling.** Detect whether the burn-in itself is stable, or use a reference period chosen with labels, so that stream normalisation also works when drift has begun before monitoring.
2. **Calibrated harm alerting.** Control the false-alert rate over a whole monitoring period (for example with sequential testing) instead of per batch, so that per-episode alarm targets become achievable.
3. **Richer density-ratio estimation.** Combine importance weighting with uncertainty estimates, and test other models than logistic regres-

sion as the monitored model.

4. **Label changes.** Use the few labels that arrive early (partial labeling) to detect changes in $P(Y \mid X)$, which no input-only signal can see.

5. **More data types.** Extend the benchmark to multi-class, text and image data, and to real temporal streams with documented drift.

6. **Deployment.** Package the two tiers as a service that reads batches from a production pipeline, with the dashboard as its operator view.

7. **Public pre-registration.** Register future experiments on a public registry before running them.

Appendix A

Pre-Registered Hypotheses

Every hypothesis below was committed to the repository, with its pass rule, before the data that judged it were downloaded. The overall verdict rule of each experiment was fixed at the same time.

ID	Claim	Pass rule	Result
v6 (held-out datasets; success if H1 and H3 pass)			
H1	Predictive warning for gradual harmful shifts	T1 $\geq$ 0.60, median lead $\geq$ 2, FA $\leq$ 5	Fail
H2	Earlier than waiting for labels	T2 gain $\geq$ 0.10, paired CI $>$ 0	Fail
H3	Separates harmful from harmless shifts	AUROC $\geq$ 0.80, harmless $\leq$ 0.10	Fail
H4	Beats the original DEWS score	higher T1 and T2 on $\geq$ 4 of 5 datasets	Fail
H5	Carries over to natural streams	T2 gain over labels $\geq$ 0	Fail
v7 (six fresh datasets; success if C1 and C3 pass)			
C1	WATCH warns before the accuracy drop	T1 $\geq$ 0.60, lead $\geq$ 2, FA $\leq$ 5	Pass

ID	Claim	Pass rule	Result
C2	WATCH warns before labels, better than labels	gain ≥ 0.10, CI > 0	Fail
C3	WARNING separates harmful from harmless	AUROC ≥ 0.80, harmless ≤ 0.10	Fail
C4	WATCH beats the DEWS score	higher T1 and T2 on $\geq$ 70% of datasets	Pass
C5	WATCH carries over to natural streams	T2 gain over labels ≥ 0	Fail
v8 (five fresh datasets; success if D1 and D2 pass)			
D1	HARM separates harmful from harmless	AUROC ≥ 0.80	Fail
D2	HARM rarely alerts on harmless shifts	harmless ≤ 0.10, FA $\leq$ 5	Fail
D3	HARM cuts harmless alarms vs WATCH while warning	drop ≥ 0.30, T2 ≥ 0.50, FA ≤ 5	Pass
D4	HARM ranks harm better than WATCH	gain ≥ 0.10, CI > 0	Pass
v9 (development gate)			
G9	A nonlinear estimator reaches the target	development AUROC $\geq$ 0.80	Fail
v10 (six fresh datasets; all three required)			
E1	Few harmless alarms	harmless ≤ 0.15	Pass
E2	Still warns before labels	T2 ≥ 0.50	Fail
E3	Within the false-alert budget	FA ≤ 5	Pass

Appendix B

Datasets

All datasets were obtained from OpenML [29]. For multi-class targets the majority class is used against the rest; the first 20 non-constant features are used.

Group	Datasets (OpenML IDs where recorded in the protocol)
v6 development	covertype, CovPokElec, Hyperplane, KDDCup99, LED, poker-hand, run-or-walk, vehicle-SensIT
v6 held-out	airlines, bank-marketing, electricity, ldpa, SEA (drawn with `default_rng(2026)`)
v7 confirmation	adult (1590), MiniBooNE (41150), higgs (23512), nomao (1486), numerai28.6 (23517), Click_prediction_small (1220)
v8 confirmation	default-of-credit-card-clients (42477), kick (41162), house_16H (821), 2dplanes (727), mv (881); albert (41147) could not be downloaded and was dropped before any episode was built
v10 confirmation	road-safety (42803), Diabetes130US (4541), jannis (41168), rl (41160), fried (901), jungle_chess (41027)

Group	Datasets (OpenML IDs where recorded in the protocol)
Natural streams	328 non-overlapping row-order windows of the 13 v6 datasets (from studies v2–v4)

Recorded deviations. MiniBooNE (v7) and rl (v10) store their rows sorted by class, so no contiguous block contains both classes; each was shuffled once with a fixed seed before any confirmation episode was built.

Appendix C

Reproducing the Results

C.1 Environment

```
python -m venv .venv
.venv\Scripts\activate          # Linux: source .venv/bin/activate
pip install -r requirements-lock.txt
```

C.2 Verdicts, ablations and post hoc analyses

```
python scripts/reproduce.py --dry-run   # show the plan
python scripts/reproduce.py             # rebuild and verify
python scripts/run_ablations.py
python scripts/run_posthoc.py
```

C.3 Dashboard, tests, figures

```
streamlit run streamlit_app.py
pytest -q
python scripts/make_paper_figures.py
python report/figures/make_report_charts.py
python report/figures/drawio/build_diagrams.py
python report/figures/drawio/export_drawio.py
```

C.4 Frozen files

Experiment	File	SHA-256 (prefix)
v6	`artifacts/v6/frozen_choices.json`	93019702…e4ad
v6	`artifacts/v6/frozen_calibrators.pkl`	93f58147…409e
v7	`artifacts/v7/frozen_v7.json`	4a9ab970cb58fad9
v7	`artifacts/v7/frozen_v7_calibrator.pkl`	dab467db1cc2045e
v8	`artifacts/v8/frozen_v8.json`	b8005ca95d70cc4f
v10	`artifacts/v10/frozen_v10.json`	c30860f3adda1561

Bibliography

[1] Albert Bifet and Ricard Gavaldà. Learning from time-changing data with adaptive windowing. In *Proceedings of the SIAM International Conference on Data Mining (SDM)*, 2007.

[2] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. The ML test score: A rubric for ML production readiness and technical debt reduction. In *IEEE International Conference on Big Data*, 2017.

[3] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[4] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[5] Evidently AI. Evidently: An open-source framework to evaluate, test and monitor ML systems. `https://github.com/evidentlyai/evidently`. Accessed November 2026.

[6] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

[7] João Gama, Pedro Medas, Gladys Castillo, and Pedro Rodrigues. Learning with drift detection. In *Advances in Artificial Intelligence – SBIA 2004*, volume 3171 of *Lecture Notes in Computer Science*, pages 286–295. Springer, 2004.

[8] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014.

[9] Saurabh Garg, Sivaraman Balakrishnan, Zachary C. Lipton, Behnam Neyshabur, and Hanie Sedghi. Leveraging unlabeled data to predict out-of-distribution performance. In *International Conference on Learning Representations (ICLR)*, 2022.

[10] Arthur Gretton, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13:723–773, 2012.

[11] Maciej Grzenda, Heitor Murilo Gomes, and Albert Bifet. Delayed labelling evaluation for data streams. *Data Mining and Knowledge Discovery*, 34:1237–1266, 2020.

[12] Devin Guillory, Vaishaal Shankar, Sayna Ebrahimi, Trevor Darrell, and Ludwig Schmidt. Predicting with confidence on unseen distributions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[13] Yiding Jiang, Vaishnavh Nagarajan, Christina Baek, and J. Zico Kolter. Assessing generalization of SGD via disagreement. In *International Conference on Learning Representations (ICLR)*, 2022.

[14] Zachary C. Lipton, Yu-Xiang Wang, and Alexander J. Smola. Detecting and correcting for label shift with black box predictors. In *International Conference on Machine Learning (ICML)*, 2018.

[15] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, João Gama, and Guangquan Zhang. Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12):2346–2363, 2019.

[16] Frank J. Massey. The Kolmogorov–Smirnov test for goodness of fit. *Journal of the American Statistical Association*, 46(253):68–78, 1951.

[17] NannyML. Performance estimation: Confidence-based performance estimation (CBPE). Documentation, `https://nannyml.readthedocs.io/en/v0.10.3/how_it_works/performance_estimation.html`, 2023.

[18] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.

[19] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.

[20] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[21] Joaquin Quiñonero-Candela, Masashi Sugiyama, Anton Schwaighofer, and Neil D. Lawrence, editors. *Dataset Shift in Machine Learning*. MIT Press, 2009.

[22] Stephan Rabanser, Stephan Günnemann, and Zachary C. Lipton. Failing loudly: An empirical study of methods for detecting dataset shift.

In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[23] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[24] Hidetoshi Shimodaira. Improving predictive inference under covariate shift by weighting the log-likelihood function. *Journal of Statistical Planning and Inference*, 90(2):227–244, 2000.

[25] Streamlit Inc. Streamlit: A faster way to build and share data apps. `https://streamlit.io`. Accessed November 2026.

[26] Masashi Sugiyama, Matthias Krauledat, and Klaus-Robert Müller. Covariate shift adaptation by importance weighted cross validation. *Journal of Machine Learning Research*, 8:985–1005, 2007.

[27] Muhammad Umer and Robi Polikar. Comparative analysis of extreme verification latency learning algorithms. arXiv:2011.14917, 2020.

[28] Emiel van Miltenburg, Chris van der Lee, and Emiel Krahmer. Preregistering NLP research. In *Proceedings of NAACL-HLT*, 2021.

[29] Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. OpenML: Networked science in machine learning. *SIGKDD Explorations*, 15(2):49–60, 2014.